

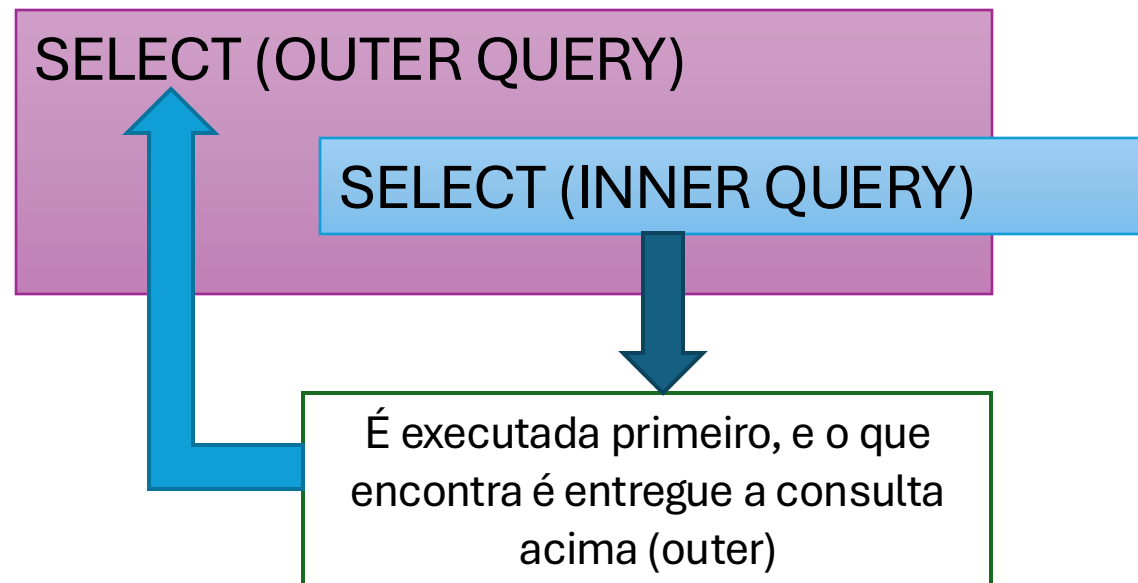
Banco de Dados

Prof^a Angelica Nakagawa Lima
angelica.lima@ibta.edu.br

SUBQUERY

Subconsulta (Subquery)

- É um comando SELECT dentro de outro para atender a uma cláusula WHERE, podendo retornar uma ou mais linhas. Podemos dizer que é uma SELECT aninhada em outra.
- Ordem de execução:



Subconsulta (Subquery)

A sintaxe básica:

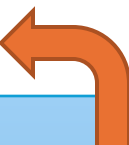
```
SELECT campos  
FROM tabela  
WHERE expressão operador  
      (SELECT campo FROM tabela);
```

Subconsulta (Subquery)

- Exemplo 1:

```
SELECT last_name, salary  
FROM hr.employees  
WHERE salary >
```

Retorno de **um
único valor**
(single row)



```
(SELECT salary  
FROM hr.employees  
WHERE UPPER(last_name) = 'ABEL');
```

Sub-consulta (Subquery)

- Exemplo 2: Listar os nomes e cargos dos funcionários com cargo igual ao do funcionário com número 141.

Sub-consulta (Subquery)

- Exemplo 2: Listar os nomes e cargos dos funcionários com cargo igual ao do funcionário com número 141.

```
SELECT last_name, job_id  
FROM hr.employees  
WHERE job_id =
```

Consulta
principal

```
(SELECT job_id  
FROM hr.employees  
WHERE employee_id = 141);
```

Subconsulta

Sub-consulta (Subquery)

- Exemplo 3: Encontrar todos os funcionários que trabalham no mesmo departamento que “David Lee”

Sub-consulta (Subquery)

- Exemplo 3: Encontrar todos os funcionários que trabalham no mesmo departamento que “David Lee”

```
SELECT first_name, last_name, job_id, department_id
FROM hr.employees
WHERE department_id=
    (SELECT department_id
     FROM hr.employees
     WHERE UPPER(first_name)= 'DAVID' AND
          UPPER(last_name)= 'LEE')
ORDER BY first_name;
```

Funções de Grupo + Single Row

- Exemplo 4: Encontrar os funcionários que ganham acima da média salarial

Funções de Grupo + Single Row

- Exemplo 4: Encontrar os funcionários que ganham acima da média salarial

```
SELECT first_name, last_name, job_id, department_id, salary  
FROM hr.employees  
WHERE salary > (SELECT AVG(salary) FROM hr.employees)  
ORDER BY first_name;
```

Sub-consulta Aninhada

- Exemplo 5: Mostrar o primeiro e último nome, cargo, cód.do departamento e salário dos empregados cujo o salário seja superior ao maior salário no departamento 'SALES'.

Sub-consulta Aninhada

- Exemplo 5: Mostrar o primeiro e último nome, cargo, cód.do departamento e salário dos empregados cujo o salário seja superior ao maior salário no departamento 'SALES'.

```
SELECT first_name, last_name, job_id, department_id, salary
FROM Employees WHERE salary >
    (SELECT MAX(salary)
     FROM hr.employees
     WHERE department_id=
        (SELECT department_id
         FROM hr.departments
         WHERE department_name='Sales')
    )
ORDER BY first_name;
```

Sub-consulta para Múltiplas Linhas

Operador EXISTS

- O operador EXISTS testa se um valor existe.(O **NOT EXISTS** garante que não existe). Se o valor existir será retornado Verdadeiro, se não existir será retornado Falso.
- Para encontrar os empregados que tem no mínimo uma pessoa subordinada a ele, faça:

```
SELECT employee_id, manager_id, first_name,  
        last_name, job_id, department_id, salary  
FROM hr.employees E  
WHERE EXISTS (SELECT employee_id  
                FROM hr.employees  
                WHERE E.manager_id = Employees.employee_id)  
ORDER BY employee_id;
```

Sub-consulta para Múltiplas Linhas

Operador NOT EXISTS

- Encontrar os empregados que não tem departamento relacionado na tabela **Departaments**:

```
SELECT employee_id, manager_id, first_name,  
        last_name, job_id, department_id, salary  
FROM hr.employees E  
WHERE NOT EXISTS (SELECT department_id  
        FROM hr.departments D  
        WHERE D.department_id= E.department_id) ;
```

Sub-consulta para Múltiplas Linhas

Operador NOT EXISTS

- Encontrar os departamentos que não tem funcionários relacionados na tabela **Employees**;

```
SELECT department_id, department_name
FROM hr.departments D
WHERE NOT EXISTS (SELECT 'X'
                    FROM hr.employees E
                    WHERE E.department_id = D.department_id)
ORDER BY 2;
```



Ordena pela segunda coluna

Sub-consulta para Múltiplas Linhas

Operador IN

- Encontrar os funcionários que trabalham nos departamentos “Marketing” e “Administração”:

```
SELECT first_name, last_name,  
        job_id, department_id, salary  
FROM hr.employees  
WHERE department_id IN (SELECT department_id  
        FROM hr.departments  
        WHERE department_name IN  
        ('Marketing','Administration'))  
ORDER BY first_name;
```

Sub-consulta para Múltiplas Linhas

Operador ANY

- Compara o valor da esquerda com cada valor retornado pela *subquery*:

< ANY = menor que o maior valor
> ANY = maior que o menor valor

```
SELECT employee_id, last_name, job_id, salary
FROM hr.employees
WHERE salary < ANY (SELECT salary
                        FROM hr.employees
                        WHERE job_id= 'IT_PROG')
AND job_id <> 'IT_PROG';
```

Neste caso esses valores são todos menores que 9000

Sub-consulta para Múltiplas Linhas

Operador ALL

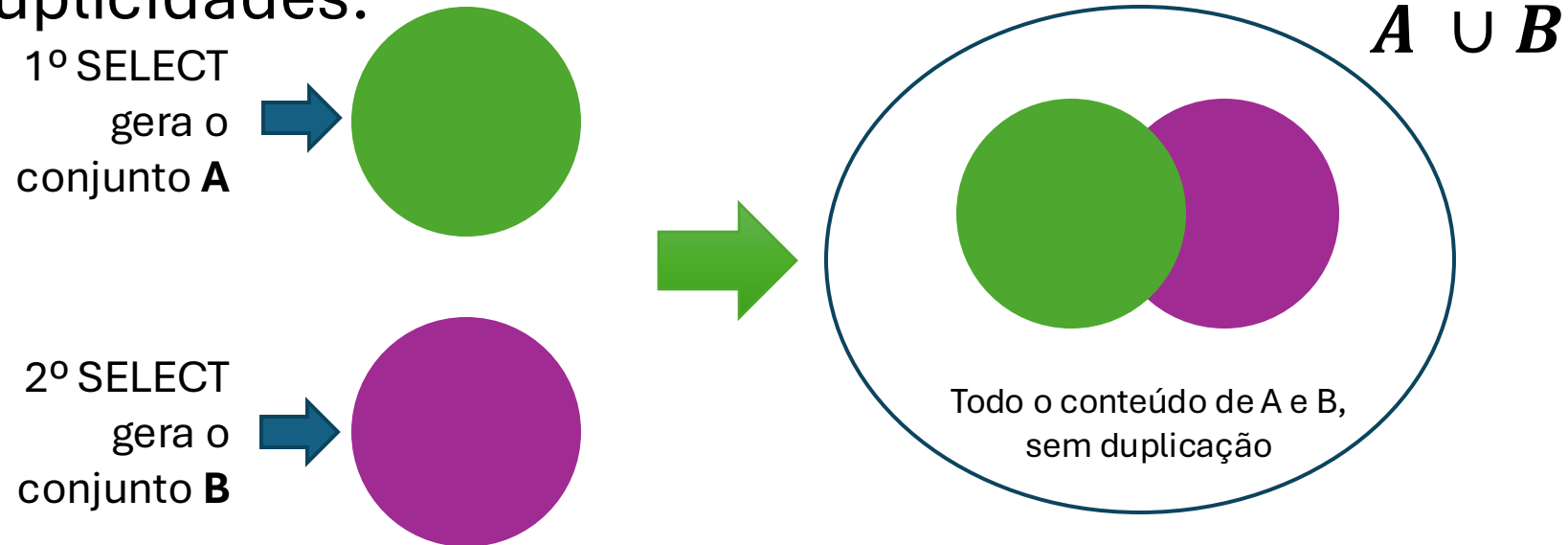
- Compara o valor da esquerda com o menor ou o maior valor retornado pela *subquery*:

```
SELECT employee_id, last_name, job_id, salary
FROM hr.employees
WHERE salary < ALL (SELECT salary
                        FROM hr.employees
                        WHERE job_id= 'IT_PROG')
AND job_id <> 'IT_PROG';
```

Neste caso esses valores são todos menores que 4200

UNION

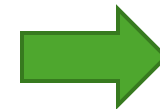
- Este operador permite combinar em um único retorno o resultado de dois SELECTs, retirando as duplicidades.



UNION - Exemplo

- Neste exemplo, conseguimos saber as funções já desempenhadas pelos funcionários e as datas em que iniciaram.

```
SELECT employee_id, job_id  
FROM employees  
UNION  
SELECT employee_id, job_id  
FROM job_history;
```



	EMPLOYEE_ID	JOB_ID
1	100	AD_PRES
2	101	AC_ACCOUNT
3	101	AC_MGR
4	101	AD_VP
5	102	AD_VP
6	102	IT_PROG
7	103	IT_PROG
8	104	IT_PROG
9	105	IT_PROG
10	106	IT_PROG

UNION - Exemplo

- Neste exemplo, conseguimos saber as funções já desempenhadas pelos funcionários.

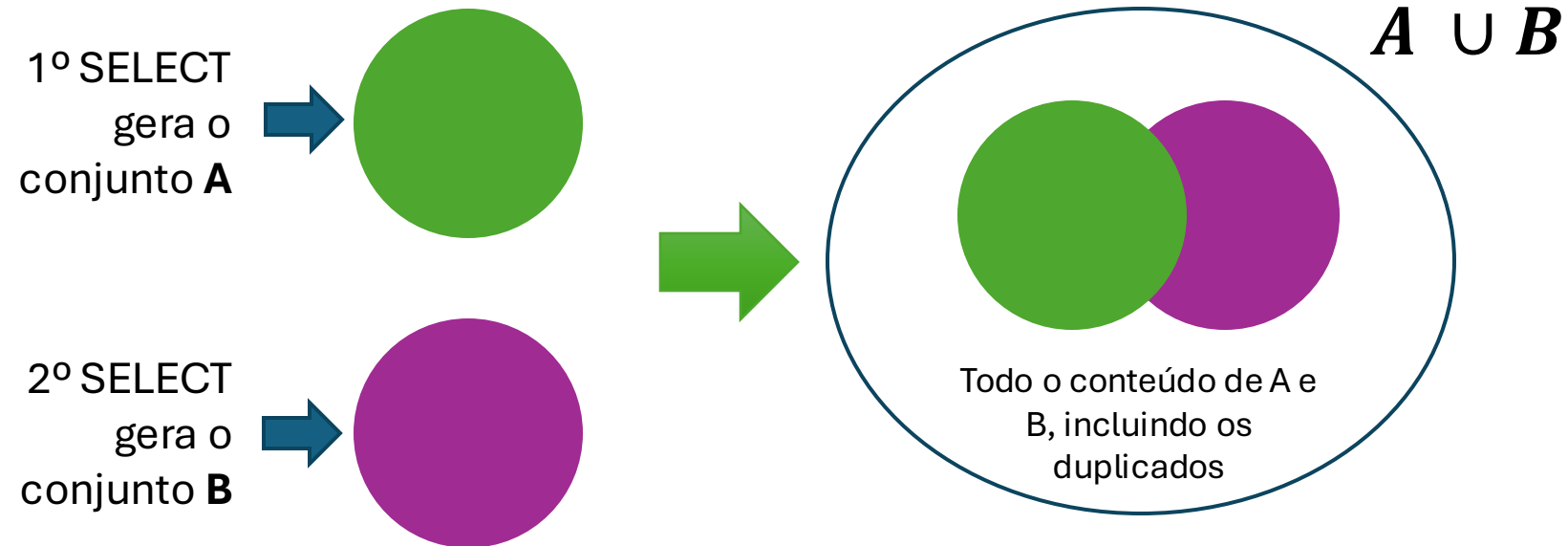
```
SELECT employee_id, job_id, TO_DATE(null) "inicio"  
FROM employees  
UNION  
SELECT employee_id, job_id, start_date  
FROM job_history;
```



	EMPLOYEE_ID	JOB_ID	inicio
1	100	AD_PRES	(null)
2	101	AC_ACCOUNT	21/09/97
3	101	AC_MGR	28/10/01
4	101	AD_VP	(null)
5	102	AD_VP	(null)

UNION ALL

- Este operador permite combinar em um único retorno o resultado de dois SELECTs, mantendo as duplicidades.



UNION ALL - Exemplo

- Neste exemplo, vamos perceber que o funcionário com o id = 176 está retornando duplicado.

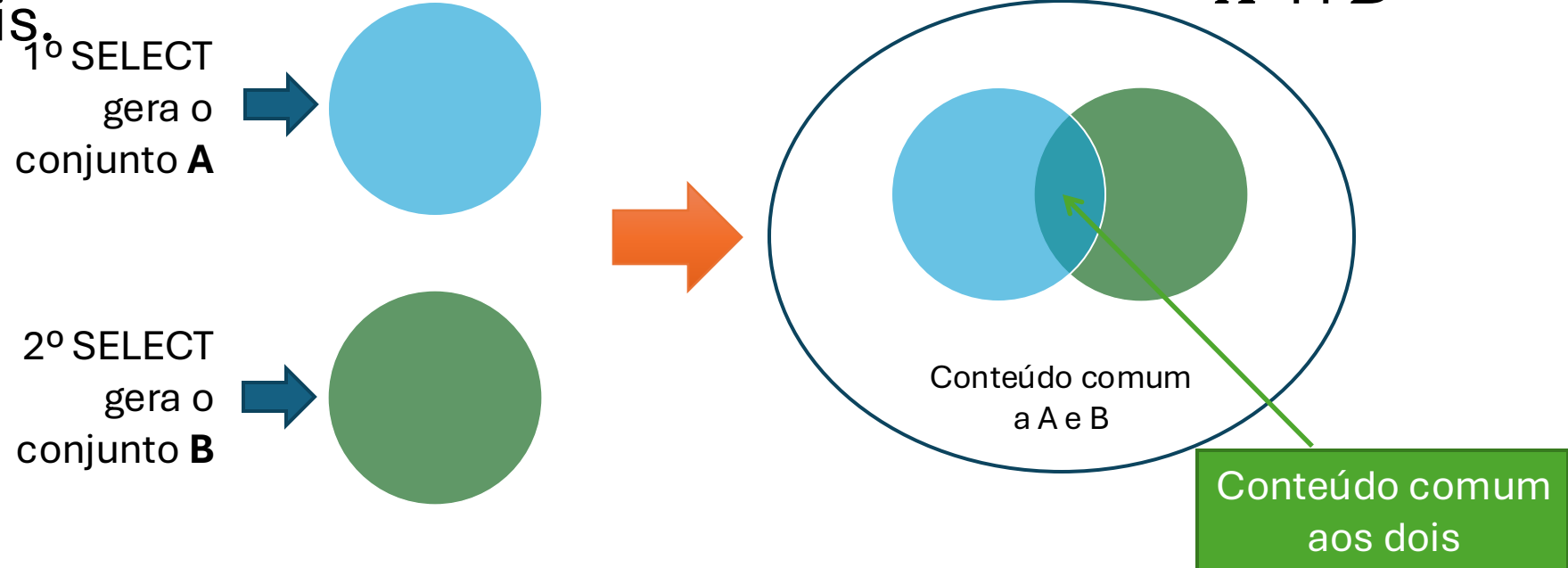
81	175	SA_REP
82	176	SA_REP
83	176	SA_MAN
84	176	SA_REP
85	177	SA_REP



```
SELECT employee_id, job_id  
FROM employees  
UNION ALL  
SELECT employee_id, job_id  
FROM job_history  
ORDER BY employee_id;
```

INTERSECT

- Este operador permite obter em um único retorno os dados comuns nos dois SELECTs envolvidos, ou seja, retorna somente o que está presente nos dois.



INTERSECT - Exemplo

- Neste exemplo, vamos perceber que são selecionados os dados que tem correspondência nas duas tabelas.

```
SELECT employee_id, job_id  
FROM employees  
INTERSECT  
SELECT employee_id, job_id  
FROM job_history  
ORDER BY employee_id;
```



	EMPLOYEE_ID	JOB_ID
1	176	SA_REP
2	200	AD_ASST

- Faça consultas nas tabelas separadamente e compare com este resultado.

INTERSECT - Exemplo

Job_history

	EMPLOYEE_ID	START_DATE	END_DATE	JOB_ID	DEPARTMENT_ID
1	101	21/09/97	27/10/01	AC_ACCOUNT	110
2	101	28/10/01	15/03/05	AC_MGR	110
3	102	13/01/01	24/07/06	IT_PROG	60
4	114	24/03/06	31/12/07	ST_CLERK	50
5	122	01/01/07	31/12/07	ST_CLERK	50
6	176	24/03/06	31/12/06	SA_REP	80
7	176	01/01/07	31/12/07	SA_MAN	80
8	200	17/09/95	17/06/01	AD_ASST	90
9	200	01/07/02	31/12/06	AC_ACCOUNT	90
10	201	17/02/04	19/12/07	MK_REP	20

Employees

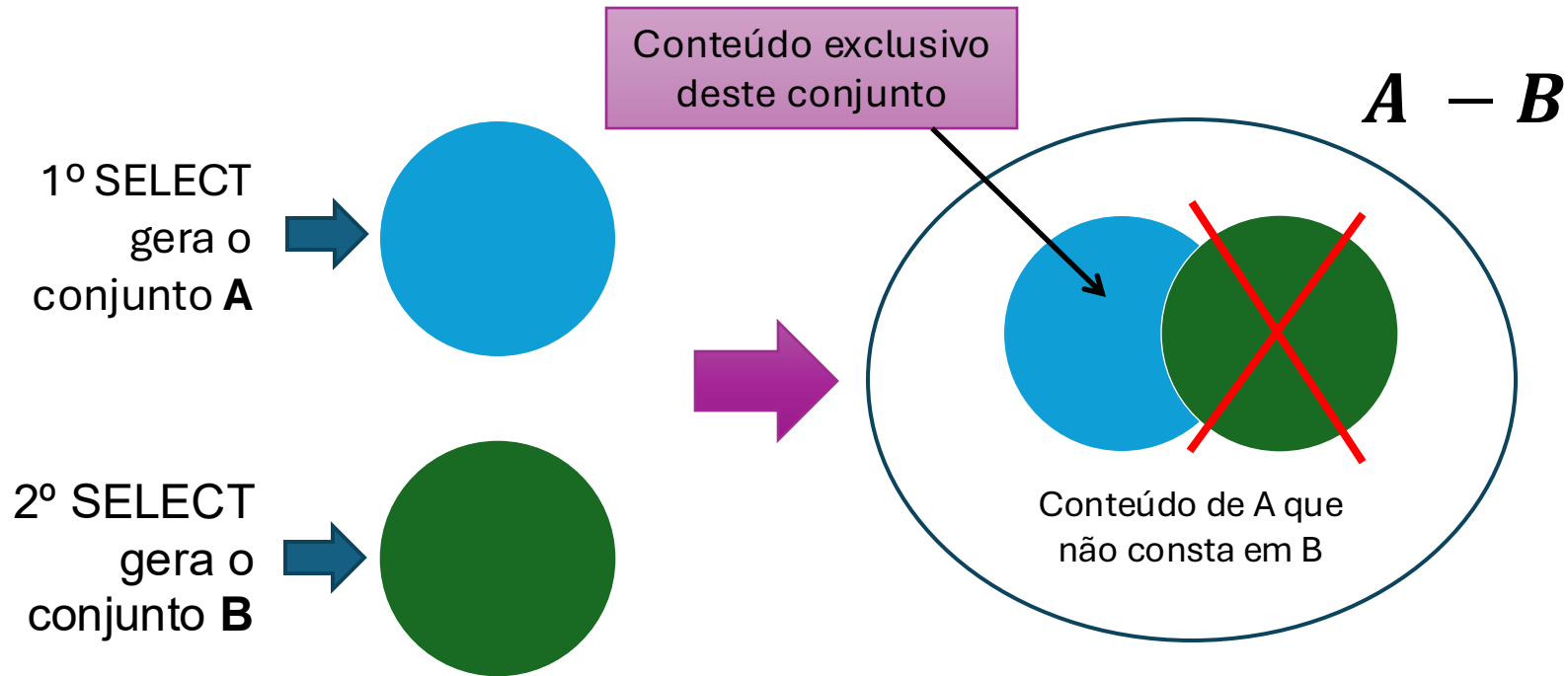
	EMPLOYEE_ID	JOB_ID
55	172	SA_REP
56	173	SA_REP
57	174	SA_REP
58	175	SA_REP
59	176	SA_REP
60	177	SA_REP
61	178	SA_REP

	EMPLOYEE_ID	JOB_ID
1	206	AC_ACCOUNT
2	205	AC_MGR
3	200	AD_ASST
4	100	AD_PRES
5	101	AD_VP
6	102	AD_VP
7	109	FI_ACCOUNT
8	...	...

- Percebe-se que há outros registros em ambas as tabelas, mas que tenham dados correspondentes em ambas, só há dois.

MINUS

- Este operador permite obter os dados consultados com a 1º **SELECT** que não constem no resultado do 2º.



- É possível consultar dados de duas tabelas diferentes, mas a quantidade e tipo das colunas precisa ser compatível.

MINUS - Exemplo

- Neste exemplo, vamos perceber que são selecionados os dados da tabela **Employee** que não constam na tabela **Job_history**.

```
SELECT employee_id, job_id
FROM employees
MINUS
SELECT employee_id, job_id
FROM job_history
ORDER BY employee_id;
```



	EMPLOYEE_ID	JOB_ID
73	172	SA_REP
74	173	SA_REP
75	174	SA_REP
76	175	SA_REP
77	177	SA_REP
78	178	SA_REP
79	179	SA_REP
80	180	SH_CLERK
81	181	SH_CLERK
82	182	SH_CLERK
83	183	SH_CLERK

INSERT usando SELECT

- É possível inserir um conjunto de registros em uma tabela utilizando um SELECT. Exemplo:

```
CREATE TABLE funcionario (  
    id_funcionario NUMBER(6) PRIMARY KEY,  
    nm_funcionario VARCHAR2(45) NOT NULL,  
    ds_email VARCHAR2(30)  
);  
SELECT * FROM funcionario;
```

```
INSERT INTO funcionario (  
    SELECT employee_id, first_name||' '||last_name, email  
    FROM employees WHERE department_id=30  
);  
DELETE FROM funcionario;
```

Criando grupos

- Anteriormente vimos o uso das funções de grupo: AVG, SUM, MAX, MIN e COUNT.
- Naquele momento as funções consideraram o conjunto de todos os registros da consulta para seu processamento, podemos dizer que este conjunto de registro é um grupo.
- Agora iremos ver como criar outro agrupamentos com base nos dados que estão sendo consultados.

Cláusula GROUP BY

```
SELECT department_id,  
        AVG(salary)  
FROM employees  
GROUP BY department_id;
```

	DEPARTMENT_ID	AVG(SALARY)
1	100	8601,3333333333333333...
2	30	4150
3	(null)	7000
4	90	19333,333333333333333...
5	20	9500
6	70	10000
7	110	10154
8	50	3475,555555555555555...
9	80	8955,8823529411764705...

- Para usarmos esta cláusula precisamos conhecer algumas regras:
 - Todos os campos do SELECT que não forem uma função de grupo devem aparecer no GROUP BY;
 - Os campos que aparecem no GROUP BY não precisam necessariamente aparecer no SELECT;

Cláusula GROUP BY

```
SELECT AVG(salary)
FROM employees
GROUP BY department_id;
```

- Podemos usar o ORDER BY

	DEPARTMENT_ID	AVG(SALARY)
1	10	4400
2	20	9500
3	30	4150
4	40	6500
5	50	3475,555555...
6	60	5760
7	70	10000
8	80	8955,882352...
9	90	19333,33333...
10	100	8601,333333...

	AVG(SALARY)
1	8601,333333...
2	4150
3	7000
4	19333,33333...
5	9500
6	10000
7	10154
8	3475,555555...
9	8955,8823529...

```
SELECT department_id, AVG(salary)
FROM employees
GROUP BY department_id
ORDER BY department_id;
```

Cláusula GROUP BY

- Podemos agrupar mais de um campo;
- Observe que são somados os salários agrupados

```
SELECT department_id "Depto", job_id "Cargo", SUM(salary) "Soma"  
FROM employees  
GROUP BY department_id, job_id  
ORDER BY department_id;
```

	Depto	Cargo	SALARY
1	10	AD_ASST	4400
2	20	MK_MAN	13000
3	20	MK_REP	6000
4	30	PU_MAN	11000
5	30	PU_CLERK	3100
6	30	PU_CLERK	2900
7	30	PU_CLERK	2800
8	30	PU_CLERK	2600
9	30	PU_CLERK	2500
10	40	HR_REP	6500

	Depto	Cargo	Soma
1	10	AD_ASST	4400
2	20	MK_MAN	13000
3	20	MK_REP	6000
4	30	PU_CLERK	13900
5	30	PU_MAN	11000
6	40	HR_REP	6500
7	50	SH_CLERK	64300
8	50	ST_CLERK	55700
9	50	ST_MAN	36400
10	60	IT_PROG	28800

Cláusula GROUP BY

- Podemos restringir os departamentos que serão considerados no SELECT usando o WHERE. Fazendo assim, eles não serão considerados na definição dos

```
SELECT department_id, ROUND(AVG(Salary))  
FROM Employees  
WHERE department_id NOT IN (10,20,30)  
GROUP BY department_id  
ORDER BY department_id;
```

	DEPARTMENT_ID	ROUND(AVG(SAL...
1	40	6500
2	50	3475,56
3	60	5760
4	70	10000
5	80	8955,88
6	90	19333,33
7	100	8601,33
8	110	10154

Cláusula HAVING

- Ela é utilizada para definir restrições dentro do agrupamento definido pelo GROUP BY, ou seja, nela definimos a condição para que os valores sejam ou não considerados dentro do grupo.
- Cuidado para não confundir com a WHERE, nesta não podemos usar funções de grupo

```
SELECT department_id, AVG(salary)  
FROM employees  
WHERE AVG(salary) > 8000  
GROUP BY department_id;
```



```
SELECT department_id, AVG(salary)  
FROM employees  
GROUP BY department_id  
HAVING AVG(salary) > 8000;
```



Cláusula HAVING

- A cláusula HAVING deve ser colocada logo após o GROUP BY;

```
SELECT department_id, MAX(salary)
FROM employees
GROUP BY department_id
HAVING MAX(salary) > 10000;
```

	DEPARTMENT_ID	MAX(SALARY)
1	100	12008
2	30	11000
3	90	24000
4	20	13000
5	110	12008
6	80	14000

	DEPARTMENT_ID	SALARY
2	110	12008
3	110	8300
4	100	12008
5	100	9000
6	100	8200
7	100	7800
8	100	7700
9	100	6900
10	90	24000
11	90	17000
12	90	17000
13	80	14000
14	80	13500
15	80	12000
16	80	11500
17	80	11000
18	80	11000
19	80	10500
20	80	10500

Cláusula HAVING

- **Outro exemplo:** mostrar a média salarial de todos os departamentos que tiverem a média maior que 6000

```
SELECT department_id,  
        AVG(salary)  
FROM Employees  
GROUP BY department_id  
HAVING AVG(salary) > 6000  
ORDER BY department_id;
```

	DEPARTMENT_ID	AVG(SALARY)
1	20	9500
2	40	6500
3	70	10000
4	80	8955,8823529411764...
5	90	19333,333333333333...
6	100	8601,333333333333...
7	110	10154
8	(null)	7000

Usando Cláusula WHERE e HAVING

- Neste exemplo é usado o WHERE para limitar a consulta aos departamentos 10, 20, 30 e 40 e o HAVING para determinar que somente serão incluídos os grupos que apresentarem os maiores salários acima de 10000.

```
SELECT department_id, MAX(salary)
FROM employees
WHERE department_id IN(10,20,30,40)
GROUP BY department_id
HAVING MAX(salary) > 10000
ORDER BY department_id;
```

```
SELECT department_id, MAX(salary)
FROM employees
WHERE department_id IN(10,20,30,40)
GROUP BY department_id
ORDER BY department_id;
```



	DEPARTMENT_ID	MAX(SALARY)
1	20	13000
2	30	11000



	DEPARTMENT_ID	MAX(SALARY)
1	10	4400
2	20	13000
3	30	11000
4	40	6500

Usando Sub-consulta

- Este exemplo mostra os códigos de departamentos e os menores salários da tabela de empregados, **agrupados por departamento**, que tenha o **menor salário maior** que o menor salário dos funcionários do **departamento 50**.

```
SELECT department_id, MIN(salary)
FROM employees
GROUP BY department_id
HAVING MIN(salary) >
    (SELECT MIN(salary)
     FROM employees
     WHERE department_id = 50);
```

	DEPARTMENT_ID	MIN(SALARY)
1	100	6900
2	30	2500
3	(null)	7000
4	90	17000
5	20	6000
6	70	10000
7	110	8300
8	80	6100
9	40	6500
10	60	4200
11	10	4400

Ordem das cláusulas no SELECT

SELECT coluna(s)

FROM tabela(s)

WHERE condição linha

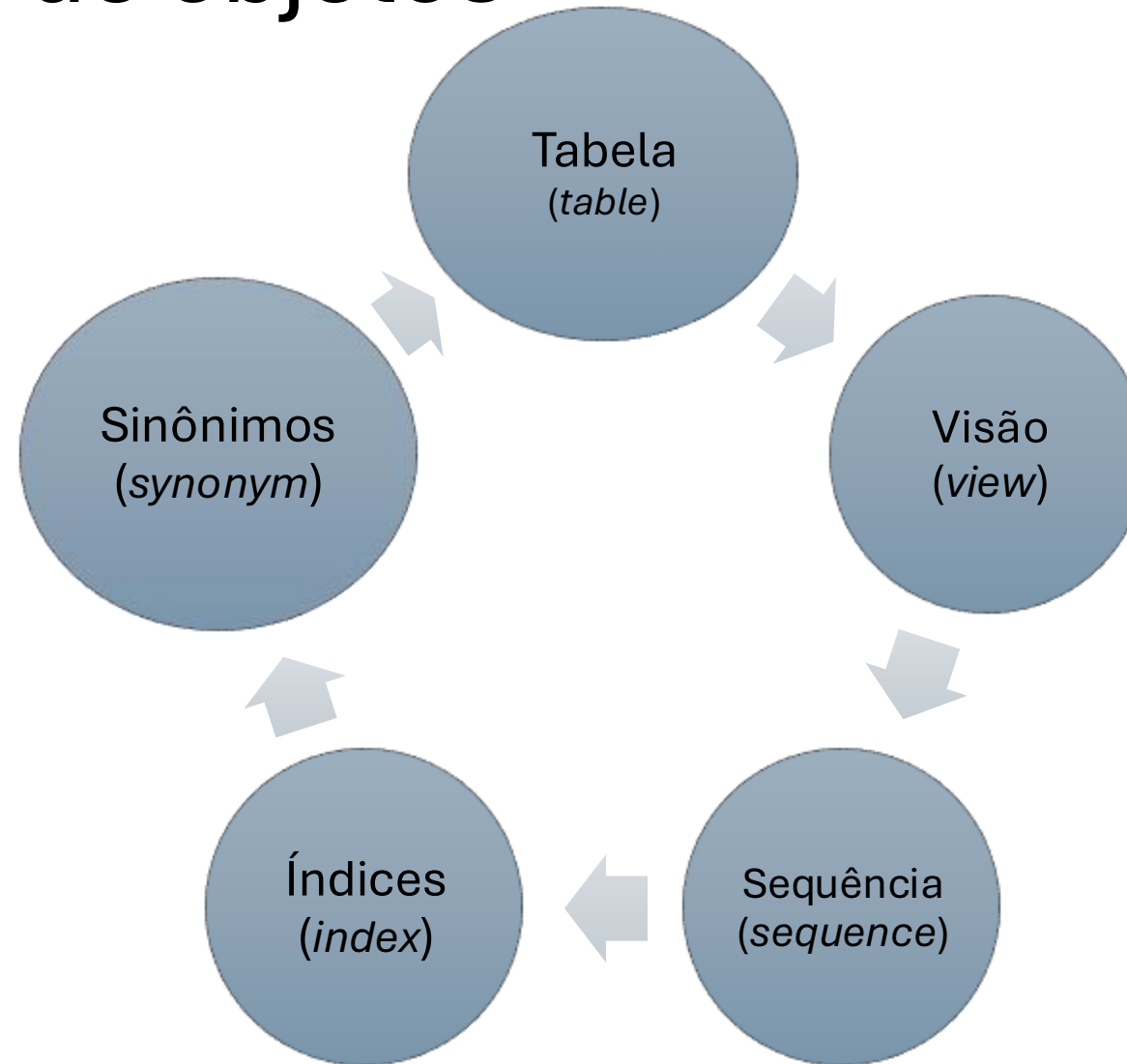
GROUP BY coluna(s)

HAVING condição de grupo de linhas

ORDER BY coluna(s);

- **WHERE** - para estabelecer condições em linhas (SELECT)
- **GROUP BY** - para configurar/definir grupos
- **HAVING** - para estabelecer condições dentro dos grupos (GROUP BY)

Tipos de objetos



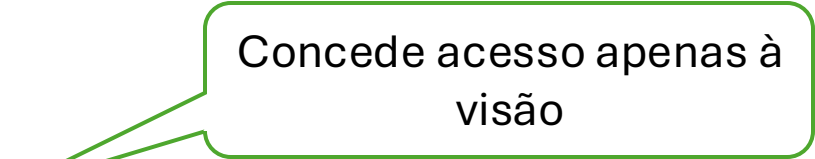
VIEW

Views (visões)

- É um tipo especial de visualização dos dados de uma ou várias tabelas;
- Ela é apenas uma visão de um grupo de colunas de uma ou mais tabelas de um grupo.
- São armazenadas no banco através de um comando select;

Vantagens:

- Restringir o acesso aos dados => pode evitar que usuários não autorizados tenham acesso a todos os dados de uma tabela (segurança);
- Independência dos dados;
- Representar diferentes visões dos mesmos dados



Concede acesso apenas à
visão

Principais vantagens do uso de views



EMP Table

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7839	KING	PRESIDENT					20
7782	CLARK	MANAGER	7839	09-JAN-81	2400		20
7934	MILLER	CLERK	7782	07-JAN-81	300		20
7900	JAMES	CLERK	7698	03-DEC-81	950		30
7521	WARD	SALESMAN	7698	22-FEB-81	1250	500	30

EMPVU10 View

EMPNO	ENAME	JOB
7839	KING	PRESIDENT
7782	CLARK	MANAGER
7934	MILLER	CLERK

Views (visões)

Sintaxe

```
CREATE VIEW nome_visão  
AS SELECT campos FROM tabela(s)  
WHERE condição
```

Criação

```
CREATE VIEW visao01  
AS SELECT employee_id, last_name, salary  
      FROM employees  
      WHERE department_id = 80;
```

Views (visões)

Com Alias

```
CREATE VIEW visao02  
AS SELECT employee_id ID_NUMBER,  
           last_name NAME, salary*12 ANN_SALARY  
FROM employees  
WHERE department_id = 50;
```

DESCRIBE visao01;

Sem suporte no Oracle live

SELECT * FROM visao02;

Mostra o conteúdo de uma
view

SELECT * FROM visao01;

Sem suporte no Oracle live

Modificando a View

Modificaremos a view **visao01** usando **CREATE OR REPLACE VIEW**. Adicionaremos apelidos, ALIAS para as colunas.

```
CREATE OR REPLACE VIEW visao01  
    (id_number, name, sal, department_id)  
AS SELECT employee_id, first_name || ' ' || last_name,  
            salary, department_id  
FROM employees  
    WHERE department_id = 80;
```



Alias

Apagando uma View

Quando apagamos uma View, perdemos os dados?

```
DROP VIEW view;
```

```
DROP VIEW empvu80;
```

Create View - complexas

Uma View com duas
tabelas
EMP e DEPT
Ligação por **FK** (deptno
da tabela EMP) e **PK**
(deptno da tabela DEPT)

Visão composta em duas tabelas (join view)

Tabelas: DEPT e EMP

Criar uma visão usando colunas de ambas as tabelas;
Como existem duas é necessário especificar o nome de
cada tabela antes do nome da coluna.

```
create view empdep_visao  
as select emp.ename, emp.deptno, dept.dname  
from scott.dept, scott.emp  
where dept.deptno = emp.deptno;
```

```
select * from empdep_visao;
```

ENAME	DEPTNO	DNAME
CLARK	10	ACCOUNTING
MILLER	10	ACCOUNTING

Create View – complexas (2)

```
Create View dep_soma
(nome, menor_sal, maior_sal, media_sal)
AS Select d.dname, min(e.sal), max (e.sal),
avg(e.sal)
From scott.emp e, scott.dept d
where e.deptno = d.deptno
Group by d.dname;
```

```
Select * from dep_soma;
```

NOME	MENOR_SAL	MAIOR_SAL	MEDIA_SAL
RESEARCH	800	3000	2175
SALES	950	2850	1566.6666666666666666666666666667
ACCOUNTING	1300	5000	2916.6666666666666666666666666667

[Download CSV](#)

3 rows selected.

Uma View com duas tabelas
EMP e DEPT
Além da utilização do
GROUP BY
Vejam que o agrupamento é
pelo nome do
departamento, pois estamos
utilizando duas tabelas. Fica
mais claro para os usuários.
(com uma tabela, o
agrupamento era pelo código)

Criando View Complexa

- Podemos criar views com *N tabelas*, e utilizando inclusive *funções de grupo*.

```
CREATE OR REPLACE VIEW dept_sum_vu  
  (name, minsal, maxsal, avgsal)  
AS SELECT  d.department_name, MIN(e.salary),  
            MAX(e.salary),AVG(e.salary)  
FROM    employees e JOIN departments d  
ON      (e.department_id = d.department_id)  
GROUP BY d.department_name;
```

JOIN

Ligações

Extraindo Dados de Mais de uma Tabela

Ligações (Joins)

Uma ligação é usada quando a pesquisa SQL requer dados de mais de uma tabela do Banco de Dados.

Linhas em uma tabela devem ser ligadas a outras linhas de acordo com o valor comum existente na coluna correspondente.

Existem três tipos principais de condições de ligações:

- 1) Equi-join
- 2) Non-equi-join
- 3) Outer-Join

Ligações

Extraindo Dados de Mais de uma Tabela

Para as próximas atividades, iremos utilizar mais de uma tabela para extrair dados.

No próximo slide, está o DER das tabelas do usuário HR no Oracle e seus relacionamentos.

Natural Join

- É baseada em todos os campos de mesmo nome em 2 tabelas.
 - Se os nomes dos campos e os tipos de dados forem iguais então retorna a consulta com sucesso;



- Se os nomes forem iguais porém os tipos de dados forem diferentes, retorna erro.



Recuperando registros: Natural Join

```
SELECT department_id, department_name,  
       location_id, city  
FROM   departments  
NATURAL JOIN locations ;
```

DEPARTMENT_ID	DEPARTMENT_NAME	LOCATION_ID	CITY
60	IT	1400	Southlake
50	Shipping	1500	South San Francisco
10	Administration	1700	Seattle
90	Executive	1700	Seattle
110	Accounting	1700	Seattle
190	Contracting	1700	Seattle
20	Marketing	1800	Toronto
80	Sales	2500	Oxford

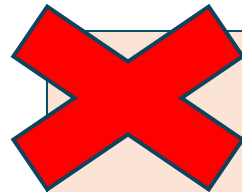
8 rows selected.

```
SELECT  department_id, department_name,  
        location_id, city  
FROM    departments  
NATURAL JOIN locations  
WHERE   department_id IN (20, 50);
```

Natural Join + USING

- Quando temos várias colunas com o mesmo nome e tipos diferentes, podemos escolher o campo de junção através do (USING).
- Não use alias de tabela ou campo no using e no where, pois lembre-se: o campo deve ser comum às duas tabelas.

```
SELECT l.city, d.department_name  
FROM   locations l JOIN departments d  
USING (location_id)  
WHERE  location_id = 1400;
```



```
SELECT l.city, d.department_name  
FROM locations l JOIN departments d USING  
(location_id)  
WHERE d.location_id = 1400;
```

EquiJoin

EquiJoin

Para descobrir, qual departamento os empregados estão, nós comparamos a coluna Department_ID da tabela Employees com o mesmo valor de Department_ID na tabela Departments. O relacionamento entre a tabela Employees e a Departments é um equi-join, em que o valor da coluna Department_ID seja igual para ambas as tabelas.

Uma condição de ligação é especificada na cláusula WHERE:

```
SELECT      coluna(s)
FROM        tabela(s)
WHERE       condição de ligação
```

Extraindo Dados de Mais de uma Tabela – Equi-Join

Para ligar as duas tabelas **Employees** e **Departments**, faça:

```
SELECT Employees.Employee_ID, Employees.Department_ID, Department_Name
```

```
FROM Employees, Departments
```

```
WHERE Employees.Department_ID = Departments.Department_ID;
```

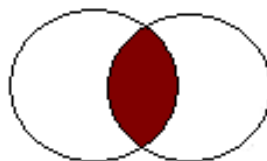
Ligação das
tabelas (JOIN)

EMPLOYEES

EMPLOYEE_ID	LAST_NAME	DEPARTMENT_ID
100	King	90
101	Kochhar	90
...		
202	Fay	20
205	Higgins	110
206	Gietz	110

DEPARTMENTS

DEPARTMENT_ID	DEPARTMENT_NAME	LOCATION_ID
10	Administration	1700
20	Marketing	1800
50	Shipping	1900
60	IT	1400
80	Sales	2500
90	Executive	1700
110	Accounting	1700
190	Contracting	1700



Inner Join

EMPLOYEE_ID	DEPARTMENT_ID	DEPARTMENT_NAME
200	10	Administration
201	20	Marketing
202	20	Marketing
...		
102	90	Executive
205	110	Accounting
206	110	Accounting

Extraindo Dados de Mais de uma Tabela – Equi-Join

Para ligar as duas tabelas Employees e Departments utilizando JOIN, faça:

```
SELECT Employees.Employee_ID,  
           Employees.Department_ID, Department_Name  
FROM   Employees JOIN Departments  
ON     Employees.Department_ID = Departments.Department_ID;
```

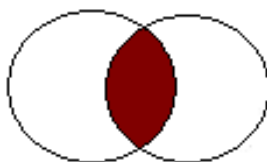
Ligação das
tabelas (JOIN)

EMPLOYEES

EMPLOYEE_ID	LAST_NAME	DEPARTMENT_ID
100	King	90
101	Kochhar	90
...		
202	Fay	20
205	Higgins	110
206	Gietz	110

DEPARTMENTS

DEPARTMENT_ID	DEPARTMENT_NAME	LOCATION_ID
10	Administration	1700
20	Marketing	1800
50	Shipping	1900
60	IT	1400
80	Sales	2500
90	Executive	1700
110	Accounting	1700
190	Contracting	1700



Inner Join

EMPLOYEE_ID	DEPARTMENT_ID	DEPARTMENT_NAME
200	10	Administration
201	20	Marketing
202	20	Marketing
...		
102	90	Executive
205	110	Accounting
206	110	Accounting

Utilizando Alias – “Apelidos”

- O ‘Alias’ também pode ser utilizado no comando select para apelidar tabelas conforme exemplo abaixo.
- O uso do alias simplifica a consulta;

```
SELECT e.employee_id, e.last_name,  
       d.location_id, department_id  
FROM   employees e JOIN departments d  
USING (department_id) ;
```

Comparação

```
SELECT Employees.Employee_ID,  
       Employees.Department_ID, Department_Name  
FROM   Employees, Departments  
WHERE  Employees.Department_ID = Departments.Department_ID
```

```
SELECT Employees.Employee_ID,  
       Employees.Department_ID, Department_Name  
FROM   Employees JOIN Departments  
       ON Employees.Department_ID = Departments.Department_ID
```

```
SELECT E.Employee_ID,  
       Department_ID, D.Department_Name  
FROM   Employees E JOIN Departments D  
       USING (department_id)
```

Ligações

Extraindo Dados de Mais de uma Tabela - Equi-Join

Utilizando Join e condições juntas:

```
SELECT E.First_Name, E.Last_Name, D.Department_Name  
FROM   Employees E, Departments D  
WHERE  E.Department_ID =D.Department_ID  
        AND E.Department_ID = 30;
```

FIRST_NAME	LAST_NAME	DEPARTMENT_NAME
Den	Raphaely	Purchasing
Alexander	Khoo	Purchasing
Shelli	Baida	Purchasing
Sigal	Tobias	Purchasing
Guy	Himuro	Purchasing
Karen	Colmenares	Purchasing

Utilizando Join e condições separadas:

```
SELECT E.First_Name, E.Last_Name, D.Department_Name  
FROM   Employees E JOIN Departments D  
ON      E.Department_ID =D.Department_ID  
WHERE  E.Department_ID = 30;
```

Ligações

Extraindo Dados de Mais de uma Tabela - Equi-Join

Caso não seja realizado o relacionamento, o SQL **não está errado mas o resultado estará errado.**

Portanto: na ausência da condição WHERE (relacionamento), cada linha da tabela Employees estará ligada com cada linha de Departments ...

Caso a tabela Employees tenha 14 registros e a tabela Departments tenha 4 linhas, será feito o **produto cartesiano (14X4=56)**, e retornará 56 registros.

A EXIBIÇÃO DESSES DADOS ESTARÁ ERRADO!!!

Self Join

EMPLOYEES (WORKER)

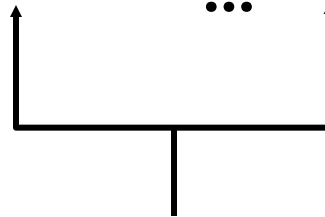
EMPLOYEE_ID	LAST_NAME	MANAGER_ID
100	King	
101	Kochhar	100
102	De Haan	100
103	Hunold	102
104	Ernst	103
107	Lorentz	103
124	Mourgos	100

...

EMPLOYEES (MANAGER)

EMPLOYEE_ID	LAST_NAME
100	King
101	Kochhar
102	De Haan
103	Hunold
104	Ernst
107	Lorentz
124	Mourgos

...



MANAGER_ID da tabela WORKER é igual ao EMPLOYEE_ID da tabela MANAGER.

Self Join

```
SELECT e.last_name emp, m.last_name mgr
FROM   employees e JOIN employees m
ON     (e.manager_id = m.employee_id);
```

Ou

```
SELECT e.last_name emp, m.last_name mgr
FROM   employees e, employees m
WHERE  e.manager_id = m.employee_id;
```

EMP	MGR
Hartstein	King
Zlotkey	King
Mourgos	King
De Haan	King
Kochhar	King
...	

19 rows selected.

Join entre très Tabelas

```
SELECT employee_id, city, department_name
FROM   employees e
JOIN   departments d
ON     d.department_id = e.department_id
JOIN   locations l
ON     d.location_id = l.location_id;
```

Ou

```
SELECT employee_id, city, department_name
FROM   employees e, departments d, locations l
WHERE  d.department_id = e.department_id AND
       d.location_id = l.location_id;
```

EMPLOYEE_ID	CITY	DEPARTMENT_NAME
103	Southlake	IT
104	Southlake	IT
107	Southlake	IT
124	South San Francisco	Shipping
141	South San Francisco	Shipping
142	South San Francisco	Shipping
143	South San Francisco	Shipping
144	South San Francisco	Shipping

••••
19 rows selected.

NonEquijoins

EMPLOYEES

LAST_NAME	SALARY
King	24000
Kochhar	17000
De Haan	17000
Hunold	9000
Ernst	6000
Lorentz	4200
Mourgos	5800
Rajs	3500
Davies	3100
Matos	2600
Vargas	2500
Zlotkey	10500
Abel	11000
Taylor	8600

...

20 rows selected.

JOB_GRADES

GRA	LOWEST_SAL	HIGHEST_SAL
A	1000	2999
B	3000	5999
C	6000	9999
D	10000	14999
E	15000	24999
F	25000	40000

← O salário dos empregados deve estar compreendido entre o menor e o maior salário da tabela JOB_GRADES.

Recuperando Registros com NonEquijoins

```
SELECT e.last_name, e.salary, j.grade_level
FROM   employees e JOIN job_grades j
ON     e.salary
      BETWEEN j.lowest_sal AND j.highest_sal;
```

Ou

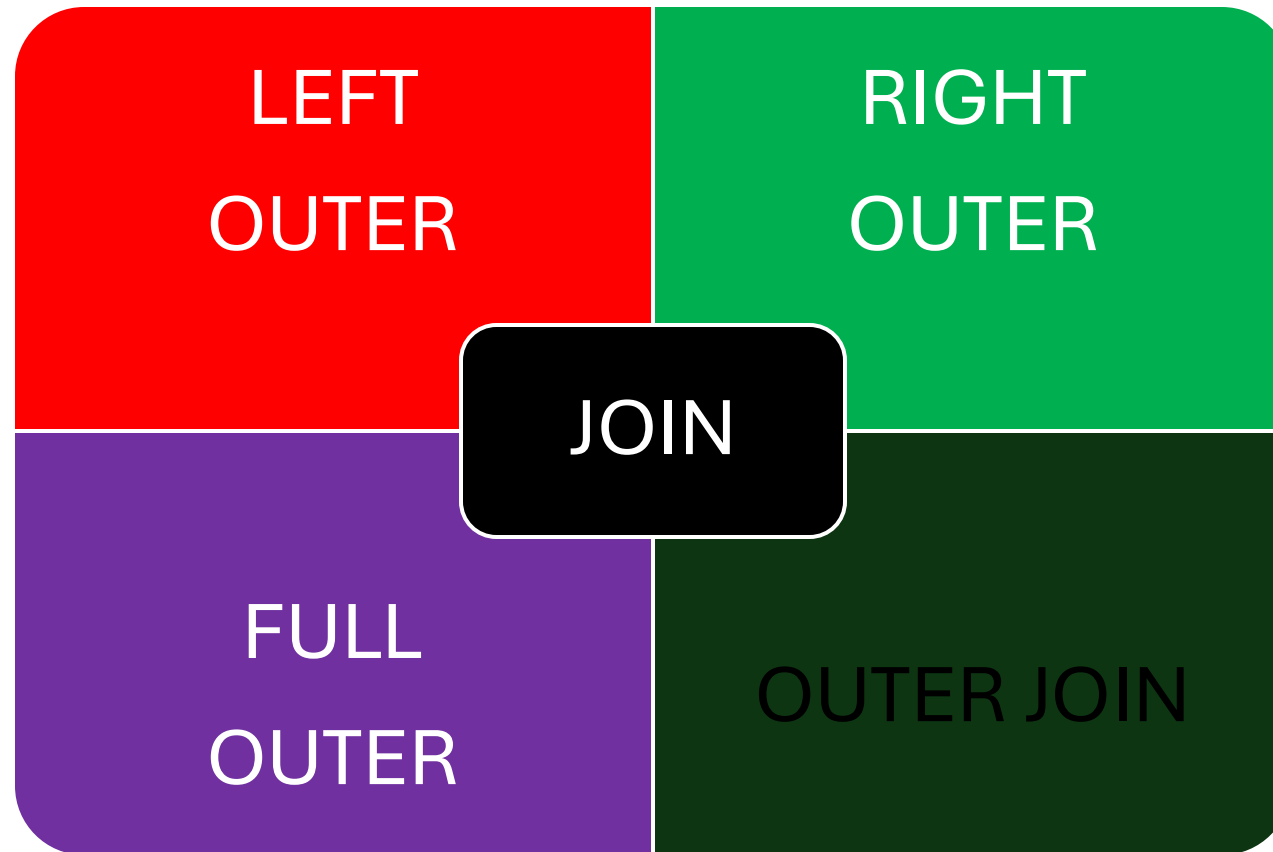
```
SELECT e.last_name, e.salary, j.grade_level
FROM   employees e, job_grades j
WHERE  e.salary BETWEEN j.lowest_sal AND
      j.highest_sal;
```

LAST_NAME	SALARY	GRA
Matos	2600	A
Vargas	2500	A
Lorentz	4200	B
Mourgos	5800	B
Rajs	3500	B
Davies	3100	B
Whalen	4400	B
Hunold	9000	C
Ernst	6000	C

...

20 rows selected.

LEFT OUTER, RIGHT OUTER e FULL OUTER JOIN



Outer Join

Além de mostrar registros cujos campos em comum estejam presentes nas duas tabelas, **ainda mostram os que faltam.**

- Neste caso, a tabela DEPARTMENTS possui o departamento de Contracting que não possui nenhum EMPLOYEES alocado.
- No caso de uma consulta na tabela de EMPLOYEES o departamento 190 é o único que não aparecerá.

DEPARTMENTS

DEPARTMENT_NAME	DEPARTMENT_ID
Administration	10
Marketing	20
Shipping	50
IT	60
Sales	80
Executive	90
Accounting	110
Contracting	190

8 rows selected.

EMPLOYEES

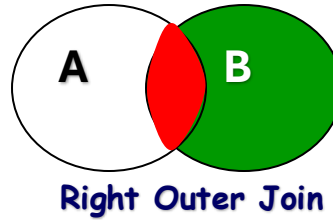
DEPARTMENT_ID	LAST_NAME
90	King
90	Kochhar
90	De Haan
60	Hunold
60	Ernst
60	Lorentz
50	Mourgos
50	Rajs
50	Davies
50	Matos
50	Vargas
80	Zlotkey

...

20 rows selected.

Não há empregados no departamento 190.

Right Outer Join



- Apresenta os departamentos que tem e que não tem funcionários.

```
SELECT e.last_name, d.department_id, d.department_name  
FROM   employees e RIGHT OUTER JOIN departments d  
ON     (e.department_id = d.department_id) ;
```

...

LAST_NAME	DEPARTMENT_ID	DEPARTMENT_NAME
Whalen	10	Administration
Fay	20	Marketing
Hartstein	20	Marketing
Davies	50	Shipping
...		
Kochhar	90	Executive
Gietz	110	Accounting
Higgins	110	Accounting
	190	Contracting

20 rows selected.

Right Outer Join (+)

Usando o Operador (+)

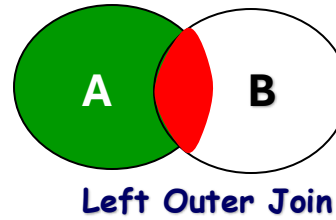
O símbolo (+) é o operador para junção externa , ou seja, se um dado existir somente em uma tabela, com este símbolo será possível retornar linha(s) faltando o outro valor.

O símbolo (+) deve ser inserido junto ao nome da tabela que não tem linhas correspondentes.

```
SELECT E.First_Name, E.Last_Name,  
       D.Department_Name  
FROM   Employees E, Departments D  
WHERE  E.Department_ID (+) =  
       D.Department_ID  
ORDER BY D.Department_Name;
```

FIRST_NAME	LAST_NAME	DEPARTMENT_NAME
William	Gietz	Accounting
Shelley	Higgins	Accounting
Jennifer	Whalen	Administration
-	-	Benefits
-	-	Construction
-	-	Contracting
-	-	Control And Credit
-	-	Corporate Tax
Lex	De Haan	Executive
Neena	Kochhar	Executive

Left Outer Join



- Apresenta os empregados que tem e que não tem departamentos.

```
SELECT e.last_name, e.department_id, d.department_name
FROM   employees e LEFT OUTER JOIN departments d
ON     (e.department_id = d.department_id) ;
```

LAST_NAME	DEPARTMENT_ID	DEPARTMENT_NAME
Whalen	10	Administration
Fay	20	Marketing
Hartstein	20	Marketing
...		
De Haan	90	Executive
Kochhar	90	Executive
King	90	Executive
Gietz	110	Accounting
Higgins	110	Accounting
Grant		

20 rows selected.

Left Outer Join

Usando o Operador (+)

```
SELECT E.First_Name, E.Last_Name, D.Department_Name  
FROM Employees E, Departments D  
WHERE E.Department_ID = D.Department_ID (+)  
ORDER BY D.Department_Name
```

FIRST_NAME	LAST_NAME	DEPARTMENT_NAME
Shelley	Higgins	Accounting
William	Gietz	Accounting
Jennifer	Whalen	Administration
Steven	King	Executive
Neena	Kochhar	Executive
Lex	De Haan	Executive
Daniel	Faviet	Finance

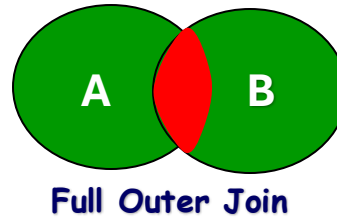
● ● ●

Ki	Gee	Shipping
Hazel	Philtanker	Shipping
Renske	Ladwig	Shipping
Stephen	Stiles	Shipping
John	Seo	Shipping
Joshua	Patel	Shipping
Trenna	Rajs	Shipping
Curtis	Davies	Shipping
Randall	Matos	Shipping
Jean	Fleaur	Shipping
Kimberely	Grant	-

106 linhas retornadas em 0,00 segundos

[Exportação](#)

Full Outer Join



- Retorna um Right e um Left Join ao mesmo tempo.

```
SELECT e.last_name, d.department_id, d.department_name
FROM   employees e FULL OUTER JOIN departments d
ON     (e.department_id = d.department_id);
```

LAST_NAME	DEPARTMENT_ID	DEPARTMENT_NAME
Whalen	10	Administration
Fay	20	Marketing
Hartstein	20	Marketing
...		
King	90	Executive
Gietz	110	Accounting
Higgins	110	Accounting
Grant		
	190	Contracting

21 rows selected.

Produto Cartesiano

EMPLOYEES (20 rows)

EMPLOYEE_ID	LAST_NAME	DEPARTMENT_ID
100	King	90
101	Kochhar	90
...		
202	Fay	20
205	Higgins	110
206	Gietz	110

20 rows selected.

DEPARTMENTS (8 rows)

DEPARTMENT_ID	DEPARTMENT_NAME	LOCATION_ID
10	Administration	1700
20	Marketing	1800
50	Shipping	1500
60	IT	1400
80	Sales	2500
90	Executive	1700
110	Accounting	1700
190	Contracting	1700

8 rows selected.

**Produto
cartesiano:
20 x 8 = 160 rows**

EMPLOYEE_ID	DEPARTMENT_ID	LOCATION_ID
100	90	1700
101	90	1700
102	90	1700
103	60	1700
104	60	1700
107	60	1700

...
160 rows selected.

Cross Join

O Cross Join retorna o produto cartesiano de 2 tabelas.

```
SELECT last_name, department_name  
FROM employees  
CROSS JOIN departments;
```

LAST_NAME	DEPARTMENT_NAME
King	Administration
Kochhar	Administration
De Haan	Administration
Hunold	Administration

...

160 rows selected.

Cross Join

O Cross Join retorna o produto cartesiano de 2 tabelas.

```
SELECT last_name, department_name  
FROM employees  
CROSS JOIN departments;
```

LAST_NAME	DEPARTMENT_NAME
King	Administration
Kochhar	Administration
De Haan	Administration
Hunold	Administration

...

160 rows selected.

Processamento de Transações

Processamento de Transações

Uma transação é um programa em execução que forma uma unidade lógica de processamento no BD;

- **Do ponto de vista do SGBD, uma transação é uma sequência de operações que são tratadas como um bloco único e indivisível (atômico) no que se refere à sua recuperação.**

Processamento de Transações

Tais operações incluem inserção, exclusão, alteração ou recuperação;

Sistemas de processamento de transações são sistemas com grandes BD e centenas de usuários executando transações concorrentes no BD.

Processamento de Transações

Exemplos: reserva de passagens, bancos, processamento de cartões de crédito, entre outros;

Um meio para especificar limites de uma transação é estabelecer explicitamente declarações de início e fim de transação;

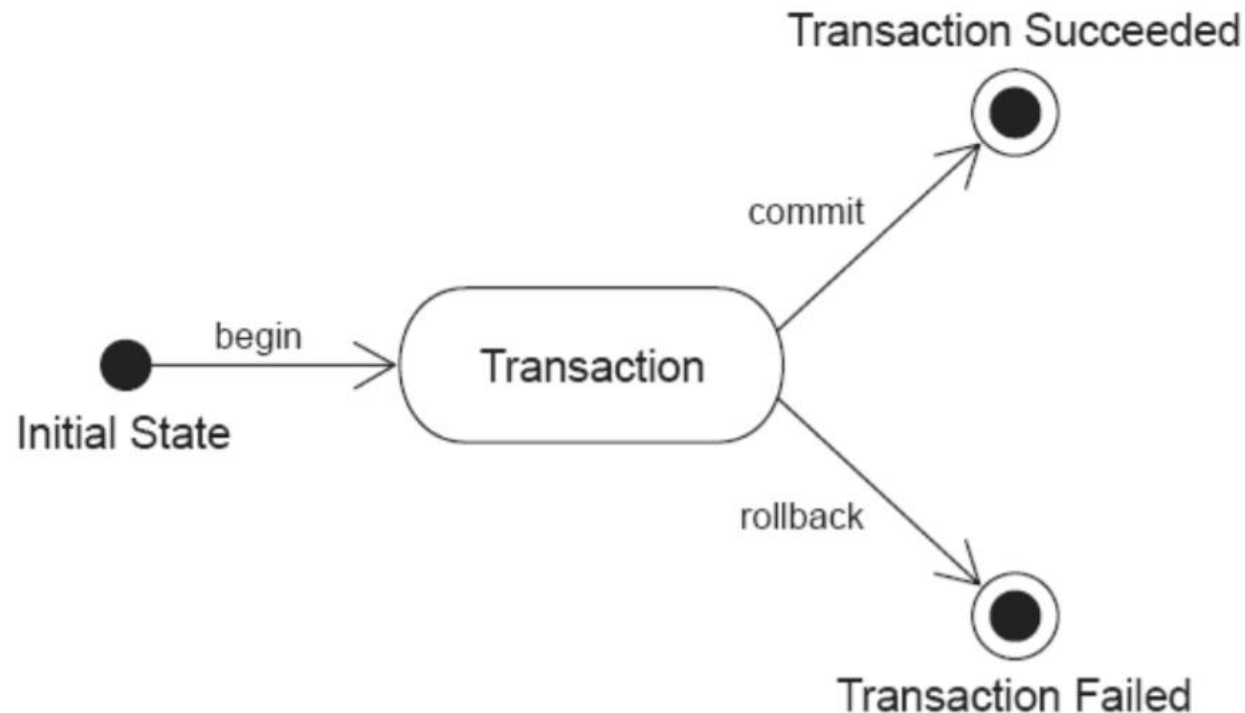
Processamento de Transações

Se uma transação for executada por um SGBD, o sistema deverá garantir que:

1. Todas as operações na transação foram completadas com sucesso e seu efeito será gravado permanentemente no BD ou;
2. A transação não terá nenhum efeito sobre o BD ou sobre quaisquer outras transações.

Processamento de Transações

Geralmente as falhas são classificadas como de transação, sistema ou mídia.



Processamento de Transações

Exemplo de transação: Seja T1 uma transação que transfere 500 reais da conta A para a conta B.

Essa transação pode ser definida como:

T1: read(A);

A := A - 500;

write(A);

read(B);

B := B + 500;

write(B);

Processamento de Transações

As transações devem possuir algumas propriedades, chamadas propriedades **ACID**, e elas devem ser impostas pelo controle de concorrência e métodos de restauração do **SGBD**.

ACID

Atomicidade **C**onsistência **I**solamento **D**urabilidade

Processamento de Transações

As propriedades **ACID** são: – **Atomicidade**: uma transação é uma unidade atômica de processamento (tudo ou nada);

Por exemplo, uma transferência de dinheiro pode ser completada ou pode falhar por várias razões, mas a atomicidade garante que uma conta não será debitada se a outra não for creditada.

Processamento de Transações

As propriedades **ACID** são: – **Consistência**:
O banco de dados deve estar num estado consistente quando a transação se inicia e quando ela termina. Isto quer dizer que uma transação não pode violar as regras ou restrições de integridade do banco de dados.

Processamento de Transações

O BD estará consistente no início e ao final da transação;

Por exemplo, se uma regra estabelece que todas as contas devem ter saldo positivo, então uma transação que viole essa regra será abortada.

Processamento de Transações

No outro exemplo, depois de executada a transação, o valor de $A + B$ deve permanecer inalterado.

Processamento de Transações

As propriedades **ACID** são: – **Isolamento**: uma transação dever ser executada como se estivesse isolada das demais.

Isto é, a execução de uma transação não deve sofrer interferência de quaisquer outras transações concorrentes;

Processamento de Transações

Cada transação não toma conhecimento de outras transações concorrentes a ela no sistema.

Implica no controle de transações.

Processamento de Transações

As propriedades **ACID** são: — **Durabilidade** ou permanência: as mudanças aplicadas ao BD por uma transação efetivada devem persistir no BD.

Essas mudanças não devem ser perdidas em razão de uma falha.

Processamento de Transações

Em geral, todas as transações são escritas num log que possa ser reexecutado de trás para frente, para recriar o sistema no seu estado correto antes da falha.

Uma transação só pode ser considerada efetivada depois de estar seguramente no log.

Processamento de Transações

Exemplo:

- Considere um sistema bancário simplificado e uma transação T1, que transfira R\$ 100,00 da conta X para a conta Y e é definida pelas operações listadas na figura a seguir.
- Considere ainda que uma transação T2 esteja sendo executada simultaneamente com T1.
- Caso a transação T2 realize a operação Escrita(Y) depois da execução da operação 4 e antes da execução da operação 6 por T1, qual propriedade de transações será violada no banco de dados do referido sistema bancário?

Processamento de Transações

T1	
1	Leitura(X);
2	X = X - 100;
3	Escrita(X);
4	Leitura(Y);
5	Y = Y + 100;
6	Escrita(Y);



Atomicidade
Isolamento
Consistência
Durabilidade

Processamento de Transações

T1	
1	Leitura(X);
2	$X = X - 100;$
3	Escrita(X);
4	Leitura(Y);
5	$Y = Y + 100;$
6	Escrita(Y);



A. Atomicidade

☒ B. Isolamento

C. Consistência

D. Durabilidade

Suporte de Transações em SQL

A definição de uma transação em SQL é similar aos conceitos de transação já definidos;

Uma única declaração SQL é sempre considerada atômica;

Suporte de Transações em SQL

O início da transação é implícito e ocorre quando declarações SQL são encontradas.

Entretanto, toda transação precisa ter uma declaração explícita de término, um **Commit** ou um **Rollback**.

Suporte de Transações em SQL

Comandos: **Commit** e **Rollback**

Além dos comandos **DML** e **DDL** vistos até aqui, uma terceira categoria de comandos, chamada **TPL** (**T**ransaction **P**rocess **L**anguage), compõe o SQL.

Suporte de Transações em SQL

Comandos: **Commit** e **Rollback**

Os comandos dessa categoria são responsáveis por gravar definitivamente no banco de dados as alterações feitas pelos comandos **DML** e **DDL** e por recuperar o banco de dados se algo de errado ocorrer.

Suporte de Transações em SQL

Comandos: **Commit** e **Rollback**

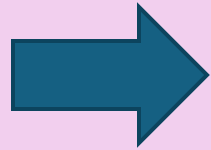
O Oracle possui dois comandos que permitem gravar os dados após uma transação bem sucedida ou recuperar a situação anterior ao início do processamento no caso de algum problema. Os comandos são **COMMIT** e **ROLLBACK**.

Suporte de Transações em SQL

- **COMMIT** - grava uma transação no banco de dados
- **ROLLBACK** - restaura uma transação. Recupera o banco de dados para a posição em que estava após o último comando COMMIT ser executado.

Suporte de Transações em SQL

COMMIT



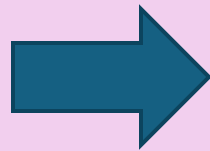
Grava fisicamente transações

Insert

Insert

Insert

ROLLBACK



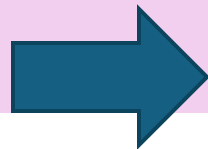
Cancela os comandos após

commit.

Insert

Insert

COMMIT



Grava fisicamente transações

Suporte de Transações em SQL

```
desc dept
```

```
select * from dept;
```

```
insert into dept values (55, 'VENDAS', 'SAO PAULO');
```

1 linha criada.

```
select * from dept;
```

```
rollback;
```

```
select * from dept;
```

Suporte de Transações em SQL

```
CREATE TABLE ValueTable (id int);  
BEGIN TRANSACTION;           -- aqui começa a transação  
    INSERT INTO ValueTable VALUES(1);  
    INSERT INTO ValueTable VALUES(2);  
COMMIT;                       -- aqui termina e "grava" tudo
```

```
-- criar tabela para testes
```

```
SELECT Nome_Livro,ISBN  
INTO Livro_ISBN  
FROM tbl_livros;
```

```
-- transação com rollback
```

```
BEGIN TRANSACTION  
    DELETE FROM Livro_ISBN -- apaga todos registros da tabela, "sem querer"  
    SELECT * FROM Livro_ISBN -- mostra tabela vazia  
ROLLBACK TRANSACTION; -- desfaz a transação  
SELECT * FROM Livro_ISBN; -- mostra os dados normalmente
```

```
-- transação com commit
```

```
BEGIN TRANSACTION  
    DELETE FROM Livro_ISBN -- apaga todos registros da tabela  
    SELECT * FROM Livro_ISBN -- mostra tabela vazia  
COMMIT TRANSACTION; -- confirma a transação  
SELECT * FROM Livro_ISBN; -- agora mostra a tabela vazia em definitivo
```

SEQUENCE

Sequence

Muitas aplicações necessitam utilizar um valor numérico sequencial como chave primária em uma tabela.

Com o uso de **Sequences** esses valores podem ser gerados automaticamente pelo Oracle.

Sequence

Uma **Sequence** gera valores para o mneumônico **nextval**, que pode ser utilizado no comando Insert.

Sequence

Comando **Create Sequence**

```
Create sequence nome  
[increment by n]  
[start with n]  
[maxvalue n |  
nomaxvalue]  
[cycle | nocycle]  
[cache n | nocache];
```

```
create sequence id_curso  
increment by 1  
start with 11;
```

Sequência criada.

Opções Sequence

Opção	Descrição
Increment by n	Especifica de quanto será o incremento ou decremento da sequência. O default é 1.
Start with n	Especifica o primeiro número a ser gerado. O default é 1.
Maxvalue n	Especifica o valor máximo que a sequência pode assumir.
Minvalue n	Especifica o valor mínimo para sequências que estejam sendo decrementadas.
Cycle Nocycle	Indica que quando atingir o valor máximo, a numeração continuará a partir do valor inicial. O default é nocycle.
Cache n Nocache	Especifica quantos valores o Oracle préaloca em memória. O default é 20.

Sequence

```
create table curso
(cod_curso number(3),
nome_curso varchar2(30) constraint curso_nome_nn not null,
carga_hor number(3),
constraint curso_cod_pk primary key(cod_curso));
```

Tabela criada.

```
desc curso
Name                                Null?    Type
-----
COD_CURSO                           NUMBER(3)
NOME_CURSO                          NOT NULL VARCHAR2(30)
CARGA_HOR                           NUMBER(3)
```

Sequence

```
insert into curso  
values (id_curso.nextval, 'Oracle Administracao', 30);
```

1 linha criada.

```
insert into curso  
values (id_curso.nextval, 'SQL Server', 30);
```

1 linha criada.

```
SQL> select * from curso;
```

Dicionário de Dados

Dicionário de Dados

O dicionário de dados (**data dictionary**) é constituído de um conjunto de tabelas que fornecem informações sobre o banco de dados.

Os principais objetos de esquema do banco de dados armazenados são:

- **tabelas, visões, índices, constraints, etc.**

Dicionário de Dados

```
SELECT object_name, object_type, created, status
FROM    user_objects
ORDER BY object_type;
```

```
SELECT table_name
FROM    user_tables;
```

```
SELECT constraint_name, constraint_type,
       search_condition, r_constraint_name,
       delete_rule, status
FROM    user_constraints
WHERE   table_name = 'EMPLOYEES';
```

Dicionário de Dados

O modelo que iremos propor é apenas uma sugestão e em outras referências ao assunto é comum encontrarmos formas diferentes de criar e manter o dicionário de dados.

Dicionário de Dados

Elabore uma tabela como no exemplo a seguir:

Entidade: Cliente				
Atributo	Classe	Domínio	Tamanho	Descrição
Codigo_cliente	Determinante	Numérico		
Nome	Simples	Texto	50	
Telefone	Multivalorado	Texto	50	Valores sem as máscaras de entrada
Cidade	Simples	Texto	50	
data_nascimento	Simples	Data		Formato dd/mm/aaaa

Analizando a tabela acima teremos:

**Entidade – Atributo – Classe –
Domínio – Tamanho – Descrição**

Dicionário de Dados

Entidade: é o nome da entidade que foi definida no MER.

A entidade é uma pessoa, objeto ou lugar que será considerada como objeto pelo qual temos interesse em guardar informações a seu respeito.

Dicionário de Dados

Atributo: Os atributos são as características da entidade Cliente que desejamos guardar.

Dicionário de Dados

Classe: as classes podem ser: simples, composto, multivalorado e determinante. Simples indica um atributo normal.

Composto indica que ele poderá ser dividido em outros atributos, como por exemplo, o endereço. Multivalorado é quando o valor do atributo poderá não ser único e determinante é um atributo que será usado como chave, como CPF, Código do cliente, etc.

Dicionário de Dados

Domínio: podem ser numérico, texto, data e booleano. Podemos chamar também de tipo do valor que o atributo irá receber.

A definição desses tipos deve seguir um processo lógico, exemplo: nome é texto, salário é numérico, data de nascimento é data e assim por diante.

Dicionário de Dados

Tamanho: define a quantidade de caracteres que serão necessários para armazenar o seu conteúdo.

Geralmente o tamanho é definido apenas para atributos de domínio texto.

Dicionário de Dados

Descrição: é opcional e pode ser usado para descrever o que é aquele atributo ou dar informações adicionais que possam ser usadas futuramente pelo analista ou programador do sistema.

Dicionário de Dados

O exemplo utilizado (**Cliente**) é referente a uma entidade, então ele deverá ser replicado para as demais entidades do seu modelo.

Entidade: Cliente				
Atributo	Classe	Domínio	Tamanho	Descrição
Codigo_cliente	Determinante	Numérico		
Nome	Simples	Texto	50	
Telefone	Multivalorado	Texto	50	Valores sem as máscaras de entrada
Cidade	Simples	Texto	50	
data_nascimento	Simples	Data		Formato dd/mm/aaaa

Stored Procedure

O que é uma Stored Procedure?

- Conjunto de comandos SQL armazenados no banco;
- Executados sob demanda;
- Podem receber parâmetros;
- Podem conter lógica (IF, LOOP, etc.)
- Analogia: "É como uma função no banco de dados"

Por que usar?

- Reutilização de código
- Segurança (não expõe SQL direto)
- Performance
- Centralização da lógica

Sintaxe básica

```
DELIMITER $$
```

```
CREATE PROCEDURE nome_procedure ()
```

```
BEGIN
```

```
-- comandos SQL
```

```
END $$
```

```
DELIMITER ;
```

Executando

```
CALL nome_procedure ( ) ;
```

Procedure com parâmetro

```
CREATE PROCEDURE busca_salario (valor DECIMAL(10,2))  
BEGIN  
    SELECT * FROM employees  
        WHERE salary > valor;  
END;
```

Listar funcionários

```
DELIMITER $$
```

```
CREATE PROCEDURE listar_funcionarios()  
BEGIN  
    SELECT * FROM employees;  
END $$
```

```
DELIMITER ;
```

```
CALL listar_funcionarios();
```

Filtro por salário

```
DELIMITER $$

CREATE PROCEDURE salario_maior(valor
DECIMAL(10,2))
BEGIN
    SELECT first_name, salary
    FROM employees
    WHERE salary > valor;
END $$

DELIMITER ;

CALL salario_maior(5000);
```

Lógica (IF)

```
DELIMITER $$

CREATE PROCEDURE classificar_salario(valor DECIMAL(10,2))
BEGIN
    IF valor < 3000 THEN
        SELECT 'Salário baixo';
    ELSEIF valor < 8000 THEN
        SELECT 'Salário médio';
    ELSE
        SELECT 'Salário alto';
    END IF;
END $$

DELIMITER ;
```

Trigger

O que é Trigger?

- Executa automaticamente
- Associado a eventos:
 - INSERT
 - UPDATE
 - DELETE

Analogia: "É como um evento automático"

Tipos

- BEFORE INSERT
- AFTER INSERT
- BEFORE UPDATE
- AFTER UPDATE
- BEFORE DELETE
- AFTER DELETE

Sintaxe básica

```
CREATE TRIGGER nome_trigger  
BEFORE INSERT ON tabela  
FOR EACH ROW  
BEGIN  
    -- ação  
END;
```

Quando usar?

- Auditoria
- Validação automática
- Logs
- Regras de negócio

Criar tabela de log

```
CREATE TABLE log_salarios (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    employee_id INT,  
    salario_antigo DECIMAL(10,2),  
    salario_novo DECIMAL(10,2),  
    data_alteracao TIMESTAMP DEFAULT  
CURRENT_TIMESTAMP  
);
```

Trigger de UPDATE

```
DELIMITER $$
```

```
CREATE TRIGGER log_update_salario  
AFTER UPDATE ON employees  
FOR EACH ROW
```

```
BEGIN  
    IF OLD.salary <> NEW.salary THEN  
        INSERT INTO log_salarios (employee_id,  
salario_antigo, salario_novo)  
        VALUES (OLD.employee_id, OLD.salary, NEW.salary);  
    END IF;  
END $$
```

```
DELIMITER ;
```

Teste

```
UPDATE employees  
SET salary = 9000  
WHERE employee_id = 2;
```

```
SELECT * FROM log_salarios;
```

Trigger de validação

```
DELIMITER $$
```

```
CREATE TRIGGER validar_salario  
BEFORE INSERT ON employees  
FOR EACH ROW  
BEGIN  
    IF NEW.salary < 0 THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Salário não pode ser negativo';  
    END IF;  
END $$
```

```
DELIMITER ;
```

NoSQL

NoSQL

NoSQL é um termo genérico que define bancos de dados não-relacionais.

A tecnologia NoSQL foi iniciada por companhias líderes da Internet - incluindo Google, Facebook , Amazon e LinkedIn - para superar as limitações (**45 anos de uso da tecnologia**) de banco de dados relacional para aplicações web modernas.

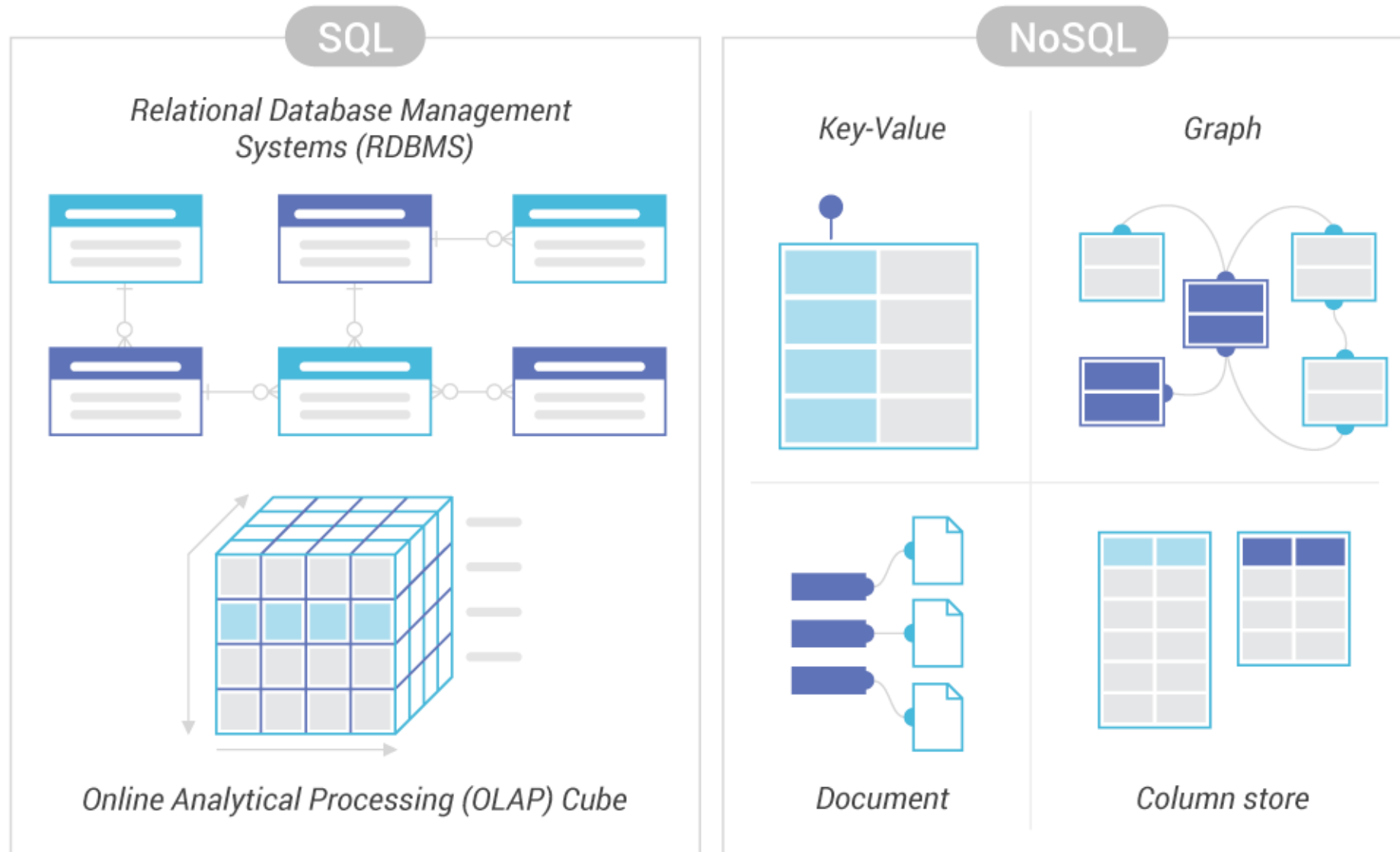
NoSQL

Quando surgiu?

Apresentado por **Carlo Strozzi**, o termo foi usado em 1998 pela primeira vez para um BD que omitiu o uso do SQL e depois seria usado novamente em 2009, em duas conferências de defensores de BDs NoSQL.



SGBDs NoSQL x SGBDs Relacionais



SGBDs NoSQL x SGBDs Relacionais

NoSQL		SQL
Modelo	Não Relacional Armazena dados em documentos “JSON”, valores em pares ou chave, grandes armazenamentos de colunas ou gráficos	Relacional Armazena dados em tabelas
Dados	Oferece flexibilidade, pois nem todo registro precisa armazenar as mesmas propriedades. Novas propriedades podem ser adicionadas instantaneamente. Os relacionamentos são frequentemente capturados desnormalizando dados e apresentando-os em um único registro. Bom para dados semiestruturados.	Ótimo para soluções em que cada registro tem as mesmas propriedades. Adicionar uma nova propriedade pode exigir a alteração de esquemas ou preenchimento de dados. Os relacionamentos são geralmente capturados em um uso de junções para resolver referências entre tabelas. Bom para dados estruturados.

SGBDs NoSQL x SGBDs Relacionais

NoSQL		SQL
Esquema	Esquemas dinâmicos ou flexíveis banco de dados em esquema-agnóstico e o esquema é ditado pelo aplicativo. Isso permite agilidade e desenvolvimento altamente iterativo	Esquema restrito O esquema deve ser mantido e sincronizado entre o aplicativo e o banco de dados
Transações	O suporte a transações ACID varia de acordo com as soluções	suporta transação ACID
Consistência	A consistência varia por solução, algumas soluções têm consistência ajustável	Suporte de consistência forte
Escala	Escala bem horizontalmente	Escala bem verticalmente

Saber quando usar ...

Use RDBMS quando...	Use NoSQL quando...
Suas aplicações forem centralizadas (ERP, CRM)	Suas aplicações forem descentralizadas (Web, Mobile, Big Data, IoT)
Alta disponibilidade moderada for necessária	Quando a disponibilidade tiver que ser contínua, sem interrupção
Dados gerados em velocidade moderada	Dados gerados em alta velocidade (sensores)
Dados forem gerados a partir de poucas fontes	Dados forem gerados a partir de múltiplas fontes
Dados forem estruturados	Dados forem semi ou não-estruturados
Transações complexas	Transações simples
For necessário manter moderado volume de dados	For necessário manter alto volume de dados

Banco de Dados Relacional

Dados são estruturados de acordo com o modelo relacional.

Padrão para a grande maioria dos SGBDs (SQL Server, Oracle, PostgreSQL, MySQL, DB2, etc.)

Elementos básicos - Relações (tabelas) e registros (tuplas)

Características fundamentais - Restrições de integridade (PK, FK, UK, CK, NN)

Normalização - Linguagem SQL (Structured Query Language)

Introdução ao NoSQL



<https://www.youtube.com/watch?v=890222oPMmM>

Principais modelos de dados em NoSQL

- Banco de dados orientado a Colunas
- Banco de dados orientado a Grafos
- Banco de dados orientado a Documentos
- Banco de dados Chave-Valor

Banco de dados orientados a Colunas

- Criada para Processar grande quantidade de dados.
- É contrário ao modelo relacional.
- Não possuem Relacionamentos.
- Não dá suporte ao uso de chaves estrangeiras.

Banco de dados orientados a Colunas

pessoa_id	pessoa_nome	pessoa_idade
12	Elis	24
13	Camila	33
14	Miguel	19

pessoa_id

12
13
14

pessoa_nome

Elis
Camila
Miguel

pessoa_idade

24
33
19

Banco de dados orientados a Colunas

- Exemplos:



A Highly Scalable NoSQL Database

Vantagens do modelo orientados a Colunas

- Altamente Escalável
- Ordenação e Manutenção de Dados
- Maior Granularidade
- Indexação

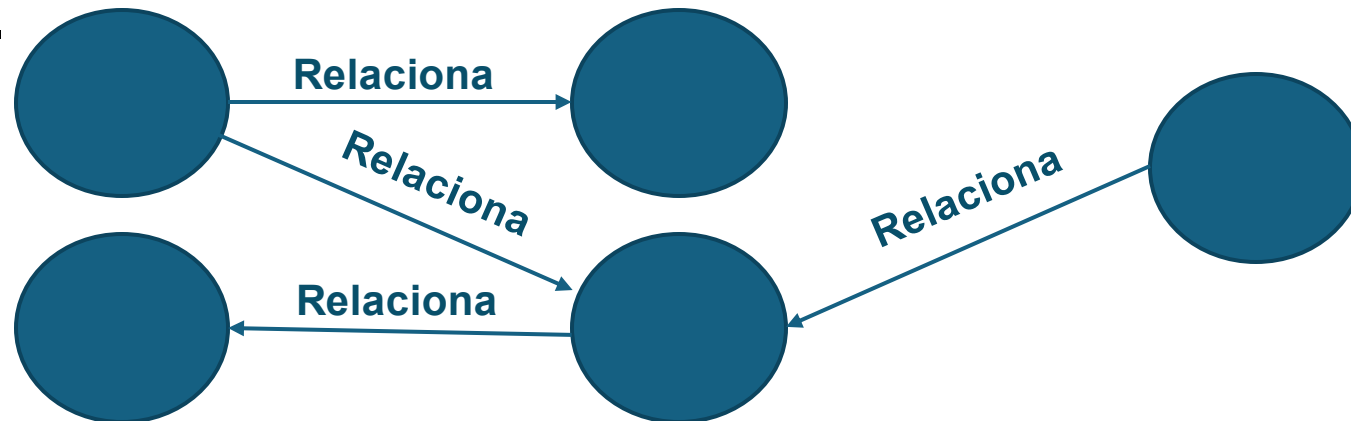
Desvantagens do modelo orientados a Colunas

- Não é tão eficiente quando se trata de pouca quantidade de dados
- Recuperação complexa de registros

Banco de dados orientados a Grafos

Em um banco de dados de grafos, relacionamentos são mais naturais.

Temos as entidades chamadas de vértices (ou node) que são ligadas entre elas pelas arestas (ou relationships) cada um podendo guardar dados entre os relacionamentos e cada relacionamento pode ter uma direção.



Banco de dados orientados a Grafos

Vértices ou Nó

- Unidade de Dados que representa uma entidade

Arestas

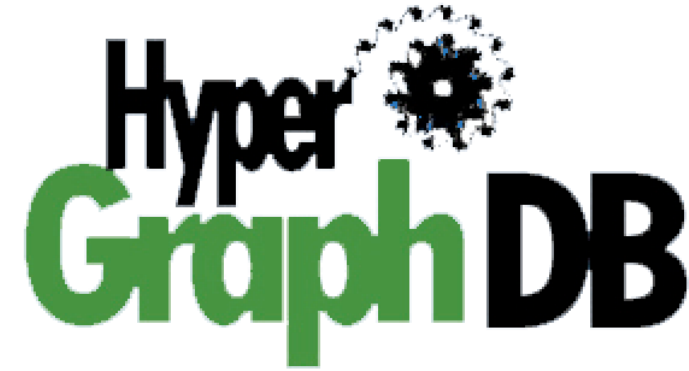
- São os relacionamentos entre as Vértices, podendo ter um Sentido

Propriedades

- Atributos das vértices e das Arestas

Banco de dados orientados a Grafos

Exemplos:



Por que NoSQL?

Hoje as empresas estão adotando NoSQL para um número crescente de casos de uso.

A escolha que é impulsionada por quatro megatendências inter-relacionadas:

- Big Users
- Big Data
- Internet das coisas
- Cloud Computing

Big Users

O crescente uso de aplicativos online resultou em um número crescente de operações de banco de dados e uma necessidade de uma maneira mais fácil de escalar bancos de dados para atender a essas demandas.

Um grande número de usuários, combinados com a natureza dinâmica dos padrões de uso está demandando uma tecnologia de banco de dados mais facilmente escalável.



3

Billion
Global Online
Population



35

Billion Hrs./Mo.
Spent Online

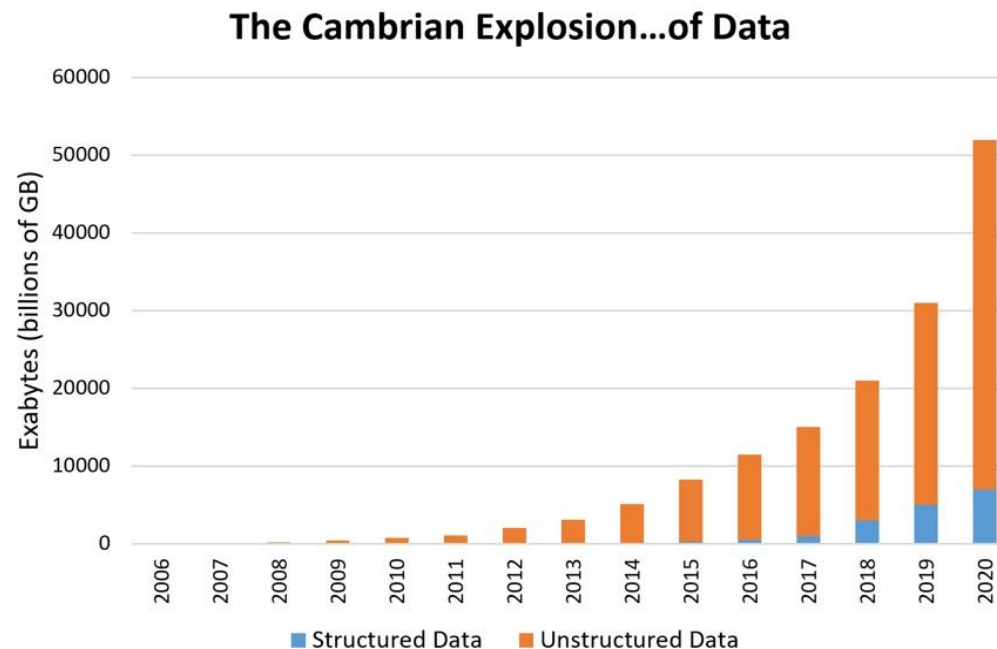


1.75

Billion
Smartphone
Users

Big Data

É necessário uma solução altamente flexível, que acomode facilmente qualquer novo tipo de dado (não-estruturado e semi-estruturado) e que não seja corrompida por mudanças na estrutura de conteúdo.



Big Data

NoSQL fornece um modelo de dados sem esquema muito mais flexível que mapeia melhor a organização de dados de uma aplicação e simplifica a interação entre a aplicação e o banco de dados, resultando em menos código para escrever, depurar e manter.

BIG DATA



A Internet das Coisas

- 32 bilhões de coisas vão estar conectadas a internet.
- 10% de todos os dados serão gerados por sistemas embarcados (vs 2% hoje)
- 21% dos mais valiosos dados serão gerados por sistemas embarcados (vs 8% hoje)



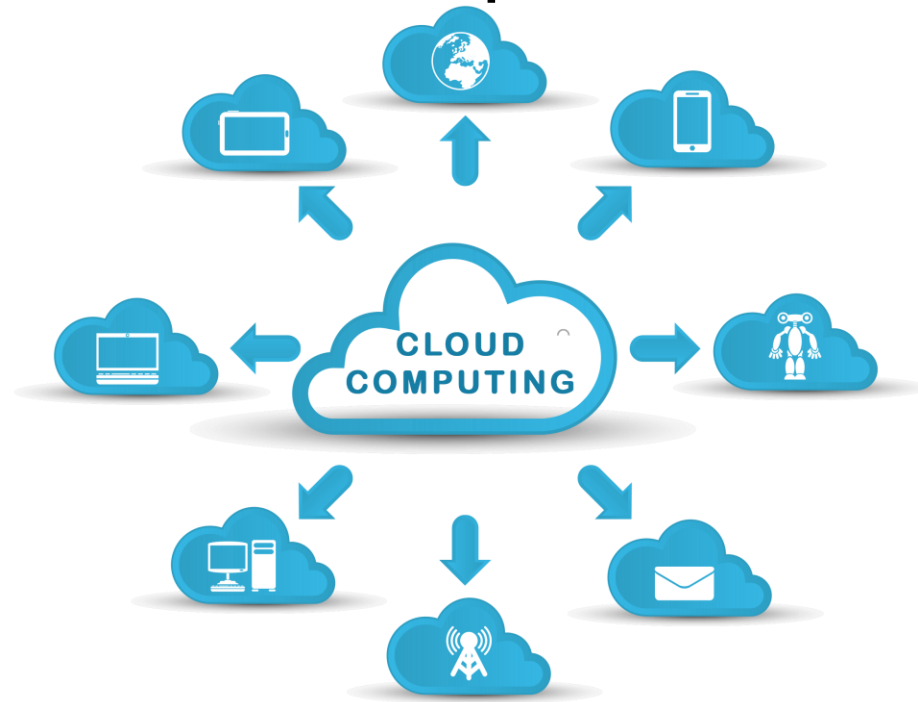
A Internet das Coisas

Dados de telemetria - semi- estruturados e contínuos - representam um desafio para bancos de dados relacionais, que exigem um esquema fixo e dados estruturados.

Empresas inovadoras estão utilizando tecnologia NoSQL para dimensionar o acesso simultâneo de dados para milhões de dispositivos e sistemas conectados, armazenar bilhões de pontos de dados e atender aos requisitos de infra-estrutura e operações de missão crítica de performance.

Cloud Computing

Atualmente a maioria das novas aplicações são executados em um sistema em nuvem privado, público ou híbrido, suportam um grande número de usuários e usam uma arquitetura de internet de três camadas.



Cloud Computing

Na camada de banco de dados, bancos de dados relacionais são originalmente a escolha popular. Seu uso é cada vez mais problemático porque eles são uma tecnologia centralizada, cuja escalabilidade é vertical ou invés de horizontal. Isso não os torna adequado para aplicações que requerem escalabilidade fácil e dinâmica.

Cloud Computing

Bancos de dados NoSQL são construídos a partir do zero para serem distribuídos, escaláveis dinamicamente e são, portanto, mais adequados a natureza altamente distribuída da arquitetura três camadas da internet.

Características NoSQL

Sistemas NoSQL possuem várias características em comum:

- Livres de esquema;
- Alta disponibilidade (Confiabilidade, recuperabilidade, detecção rápida de erros e operações contínuas);
- Escalabilidade;

Características NoSQL

Mesmo assim, possuem diversas características únicas quanto ao:

- Modo de armazenamento dos dados
- Modelo de dados

Modelo de dados mais flexível

```
{  
  "ID":1,  
  "FIRST": "Frank",  
  "LAST": "Weigel",  
  "ZIP": "94040",  
  "CITY": "MV",  
  "STATE": "CA"  
}
```

JSON

=

KEY	First	Last	ZIP_id
1	Frank	Weigel	2
2	Ali	Bob	2
3	Mark	Azad	2
4	Steve	Yen	3

+

ADDRESS INFO

ZIP_id	City	State	ZIP
1	DEN	CO	30303
2	MV	CA	94040
3	CHI	IL	60609
4	NY	NY	10010

Modo de Armazenamento de Dados

Temos os sistemas que...

- Mantêm suas informações em memória realizando persistências ocasionais - ***Scalaris, Redis***
- Mantêm suas informações em disco - **CouchDB, MongoDB, Riak, Voldemort**
- São configuráveis - **BigTable, Cassandra, Hbase, HyperTable**

Modelo de Dados

Existem quatro categorias:

- Sistemas baseados em armazenamento chave-valor
- Sistemas orientados a documentos
- Sistemas orientados à coluna
- Sistemas baseados em grafos

Modelo de Dados

KEY VALUE COLUMN GRAPH DOCUMENT

Coleção de chaves únicas associada a um valor, que pode ser de qualquer tipo (binário, string) Exemplo:
Key: 1234 Value: “Fernando” Key: 2343 Value:
“Name=Fernando, age=29”

Key	Value
123435	Joao da Silva
334545	Name=Fernando, age=29

Modelo de Dados

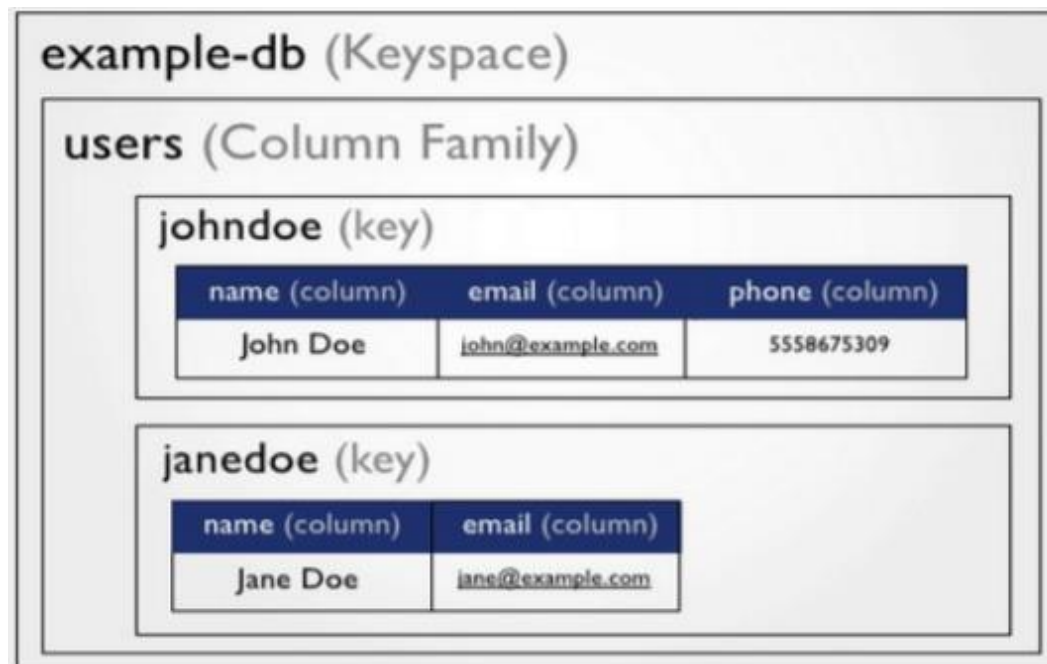
KEY VALUE **COLUMN** GRAPH DOCUMENT

Famílias de colunas (um repositório para colunas, análogo a uma tabela do Modelo Relacional) e super-colunas (compostas por arrays de colunas) o benefício de armazenar dados em colunas, é a busca /acesso rápido e a agregação de dados.

Modelo de Dados

KEY VALUE **COLUMN** GRAPH DOCUMENT

Exemplo: Column family, Key, Column name e value
Pessoas ; 4564 ; nome : Ana ; idade : 30;



Modelo de Dados

KEY VALUE COLUMN **GRAPH DOCUMENT**

Banco de dados baseado em grafos, nele temos as entidades chamadas de vértices (ou node) que são ligadas entre elas pelas arestas (ou relationships) cada um podendo guardar dados entre os relacionamentos e cada relacionamento pode ter uma direção.

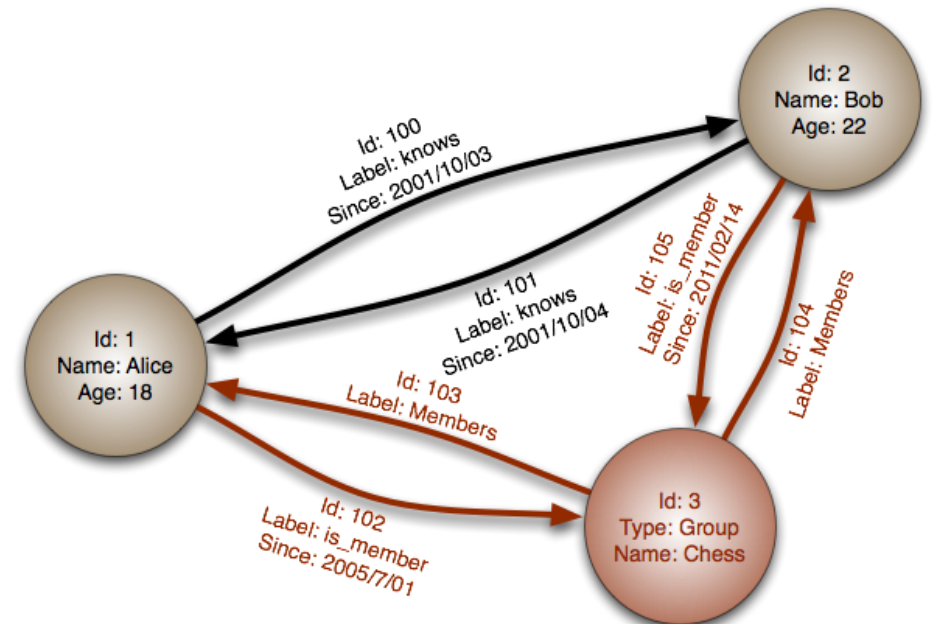
Modelo de Dados

KEY VALUE COLUMN **GRAPH** DOCUMENT

Exemplo:

- Vértice: Chave->Valor representa entidade. Nome: Alice
- Aresta: relacionamentos

Ex: Vertice “Alice” conhece o vertice “Bob” desde 2001



Modelo de Dados

KEY VALUE COLUMN GRAPH DOCUMENT

Os documentos são as unidades básicas de armazenamento e estes não utilizam necessariamente qualquer tipo de estruturação pré-definida

São baseados em JSON (JavaScript Object Notation)

Modelo de Dados

KEY VALUE COLUMN GRAPH **DOCUMENT**

Exemplo:

```
{"user":{
```

```
  "id": "123",
```

```
  "name": "Emmanuel",
```

```
  "addresses":[
```

```
    {"city":"Paris"},
```

```
    {"city":"Sao Paulo"}]]
```

Collection ("Things")

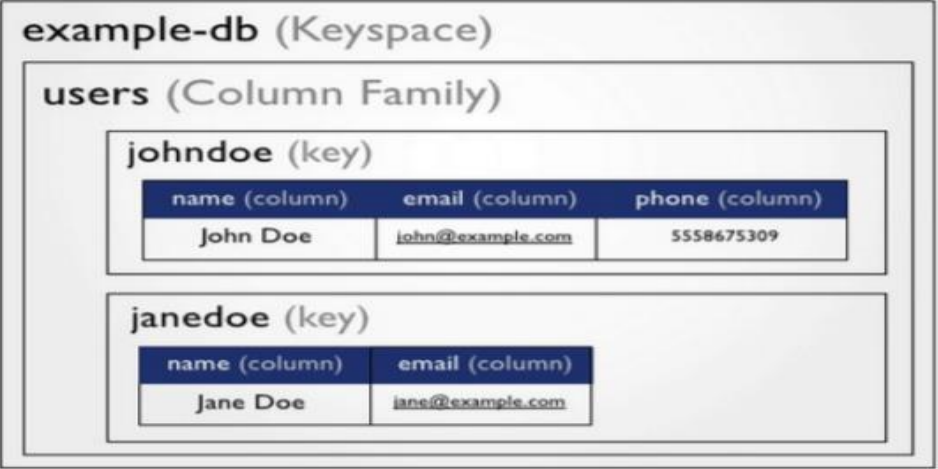


Classificação NOSQL

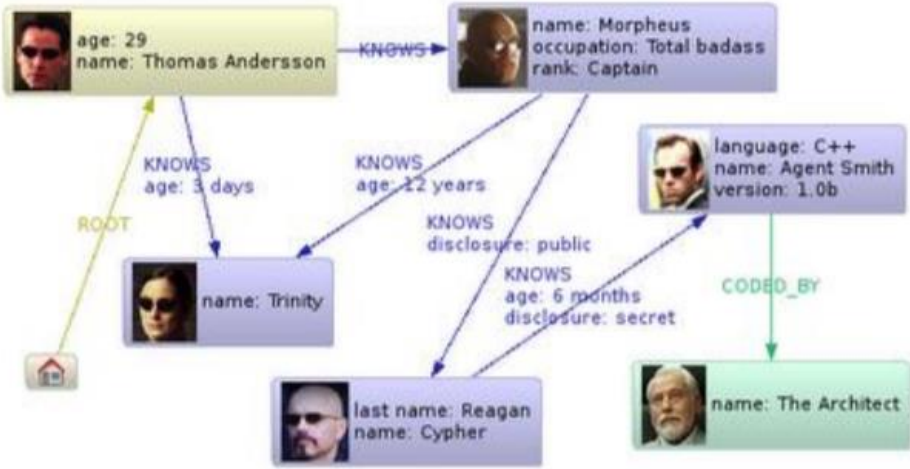
Key – Value

Key	Value
123435	Joao da Silva
334545	Name=Fernando, age=29

Column



Graph



Document



DB Ranking NOSQL

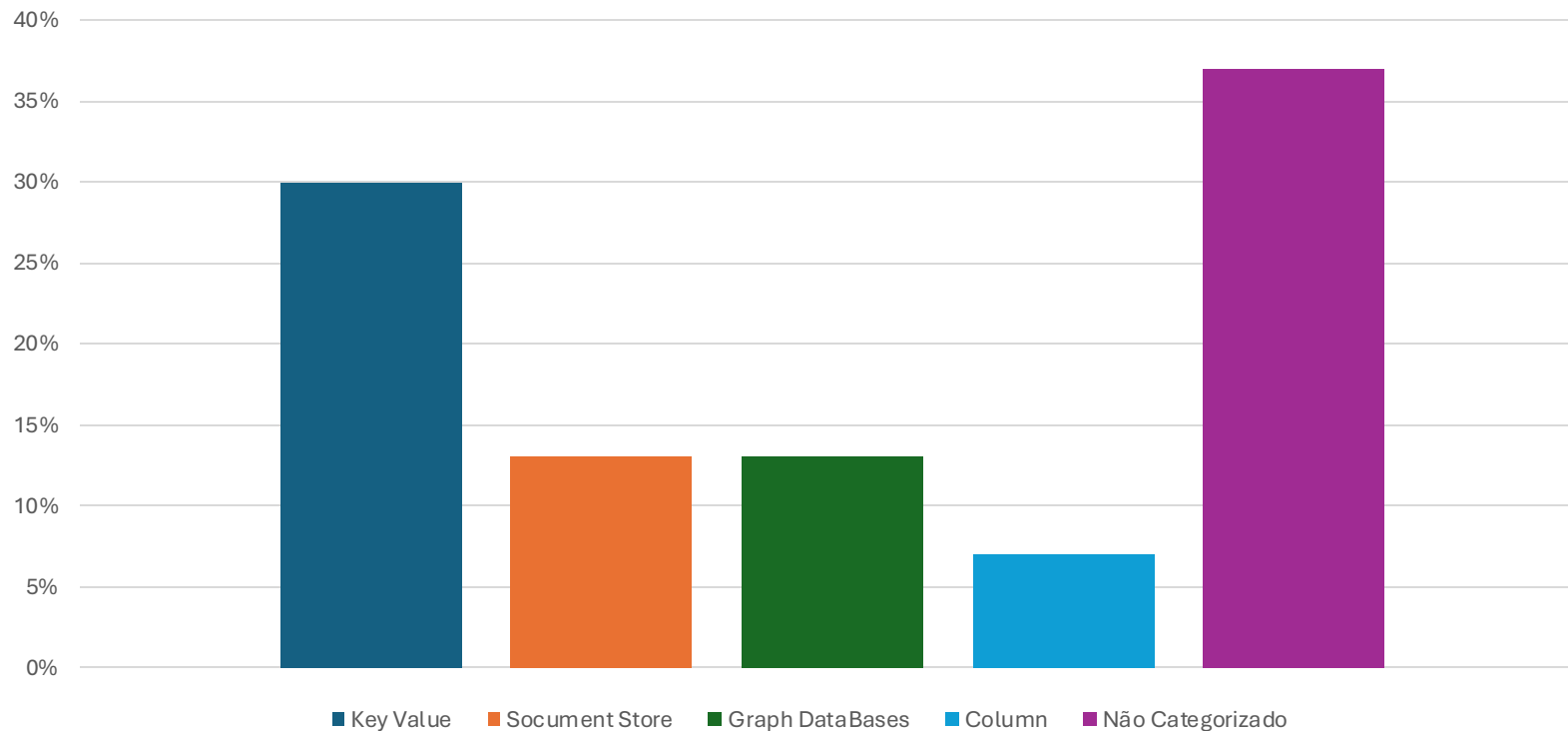
360 systems in ranking, November 2020

Rank			DBMS	Database Model	Score		
Nov 2020	Oct 2020	Nov 2019			Nov 2020	Oct 2020	Nov 2019
1.	1.	1.	Oracle +	Relational, Multi-model i	1345.00	-23.77	+8.93
2.	2.	2.	MySQL +	Relational, Multi-model i	1241.64	-14.74	-24.64
3.	3.	3.	Microsoft SQL Server +	Relational, Multi-model i	1037.64	-5.48	-44.27
4.	4.	4.	PostgreSQL +	Relational, Multi-model i	555.06	+12.66	+63.99
5.	5.	5.	MongoDB +	Document, Multi-model i	453.83	+5.81	+40.64
6.	6.	6.	IBM Db2 +	Relational, Multi-model i	161.62	-0.28	-10.98
7.	↑ 8.	↑ 8.	Redis +	Key-value, Multi-model i	155.42	+2.14	+10.18
8.	↓ 7.	↓ 7.	Elasticsearch +	Search engine, Multi-model i	151.55	-2.29	+3.15
9.	9.	↑ 11.	SQLite +	Relational	123.31	-2.11	+2.29
10.	10.	10.	Cassandra +	Wide column	118.75	-0.35	-4.47

Fonte: <https://db-engines.com/en/ranking>

Classificação de SGBDs NoSQL

150 tipos de banco de dados NOSQL

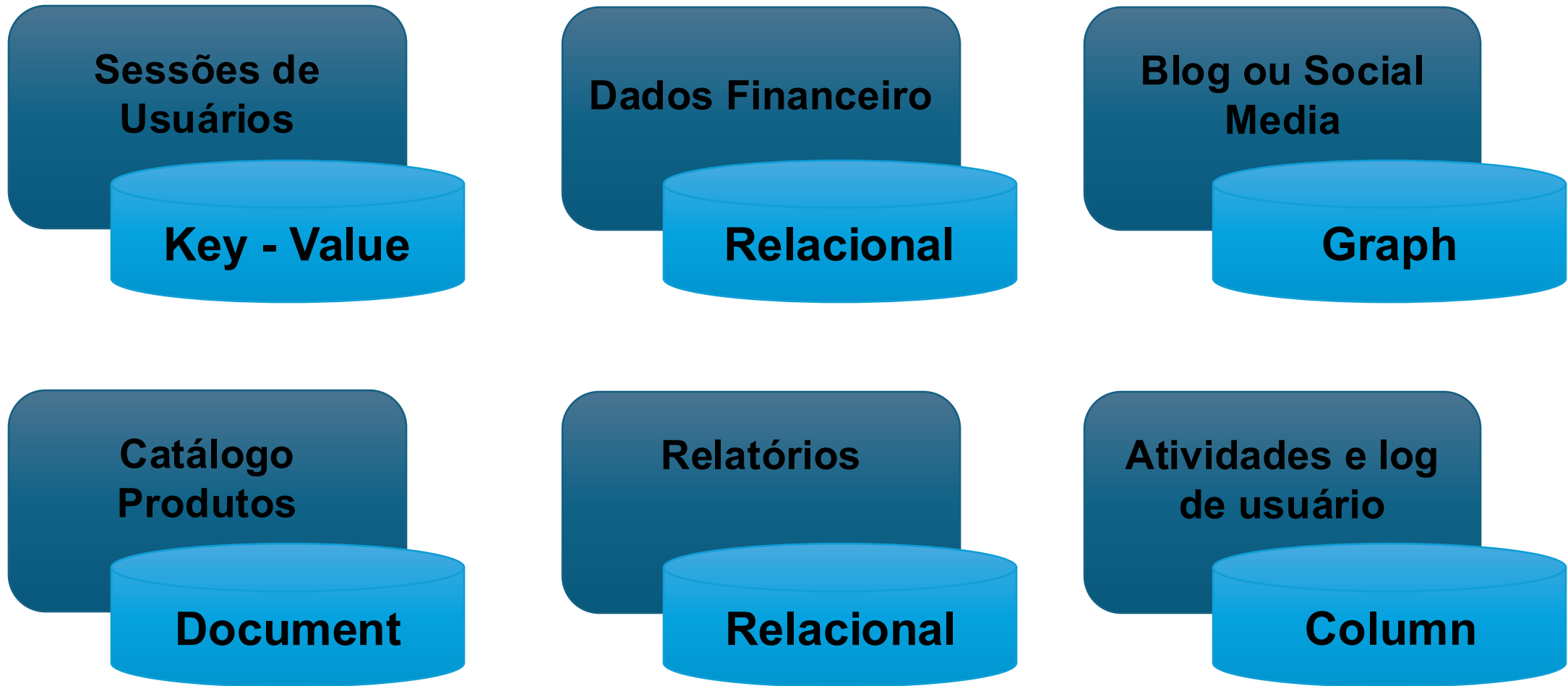


Fonte: <http://nosql-database.org/>
Dados compilados manualmente

Classificação de SGBD NOSQL e Produtos

KEY VALUE	COLUMN GRAPH	DOCUMENT	DOCUMENT
			
Amazon DynamoDB (Key-value) Desenvolvido em: Java Quem Usa: Washingtonpost.com e Elsevier (Editora)	BigTable(column) Google Desenvolvido em: C++ Quem Usa: Gmail, Google Maps e YouTube	Neo4j (graph) Desenvolvido em: Java Quem Usa: WalMart, National, Geographic e Ebay	MongoDB (Document) Desenvolvido em: C Quem Usa: Globo.com, Apontador, Forbes e New York Times
	Cassandra (column) Desenvolvido em: Java Quem Usa: Twitter, NetFlix e Facebook		

Classificação de SGBD NOSQL e Produtos



Tenho realmente que mudar?

I 
SQL

Dá pra fazer **query**?

Referência de query

Amazon DynamoDB – Key-Value

SQL Query

```
SELECT _id, name, address  
FROM users  
WHERE age > 18  
LIMIT 5
```

← projection
← table
← select criteria
← cursor modifier

AWS Query

Amazon DynamoDB Explore Table: Reply

List Tables Browse Items

☐ Scan ☒ Query Go New Item Edit Item Copy to New Delete Item

Index Name: [Table] Reply: Id, ReplyDateTime

Hash Key (*): Id equal to Amazon DynamoDB#DynamoDB Thre

Range Key: ReplyDateTime greater than 2014-01-12

Sort: ☒ Ascending ☐ Descending

Referência de query

MongoDB - Document

SQL Query

```
SELECT _id, name, address  
FROM users  
WHERE age > 18  
LIMIT 5
```

← projection
← table
← select criteria
← cursor modifier

Operation Find

```
db.users.find(  
  { age: { $gt: 18 } },  
  { name: 1, address: 1 }  
) .limit(5)
```

← collection
← query criteria
← projection
← cursor modifier

Referência de query

Neo4j - Graph

SQL Query

```
SELECT _id, name, address  
FROM users  
WHERE age > 18  
LIMIT 5
```

← projection
← table
← select criteria
← cursor modifier

Cyber Query

```
MATCH a  
WHERE a.age>18  
RETURN a.id, a.name, a.address  
LIMIT 5
```

Referência de query

Cassandra - Column

SQL Query

```
SELECT _id, name, address  
FROM users  
WHERE age > 18  
LIMIT 5
```

← projection
← table
← select criteria
← cursor modifier

CQL – Cassandra Query Language

```
SELECT _id, name, address  
FROM users  
WHERE age > 18  
LIMIT 5
```

← projection
← table
← select criteria
← cursor modifier

Comandos CRUD
(Create, Read, Update, Delete)
São Iguais

Quais são as linguagens que suportam NoSQL?

Linguagens	Amazon Dynamo	Neo4j	Cassandra	MongoDb
C				X
C#				X
C++			X	X
Go			X	X
Java	X	X	X	X
JavaScript	X			X
Node.js	X	X	X	X
Perl			X	X
PHP	X	X	X	X
Python		X	X	X
Rybu	X	X	X	X
Scala		X	X	X

O futuro dos Banco de Dados Relacional



O futuro dos Banco de Dados Relacional

Dados de 2020

Quadrante Mágico
para Plataformas
de Aprendizagem
de Máquina e
Ciência de Dados



Cases de sucesso com uso do NoSQL



SGBD: sistema de processamento de faturas mensais.

NOSQL: Sistema focado em recomendações de melhores filmes.

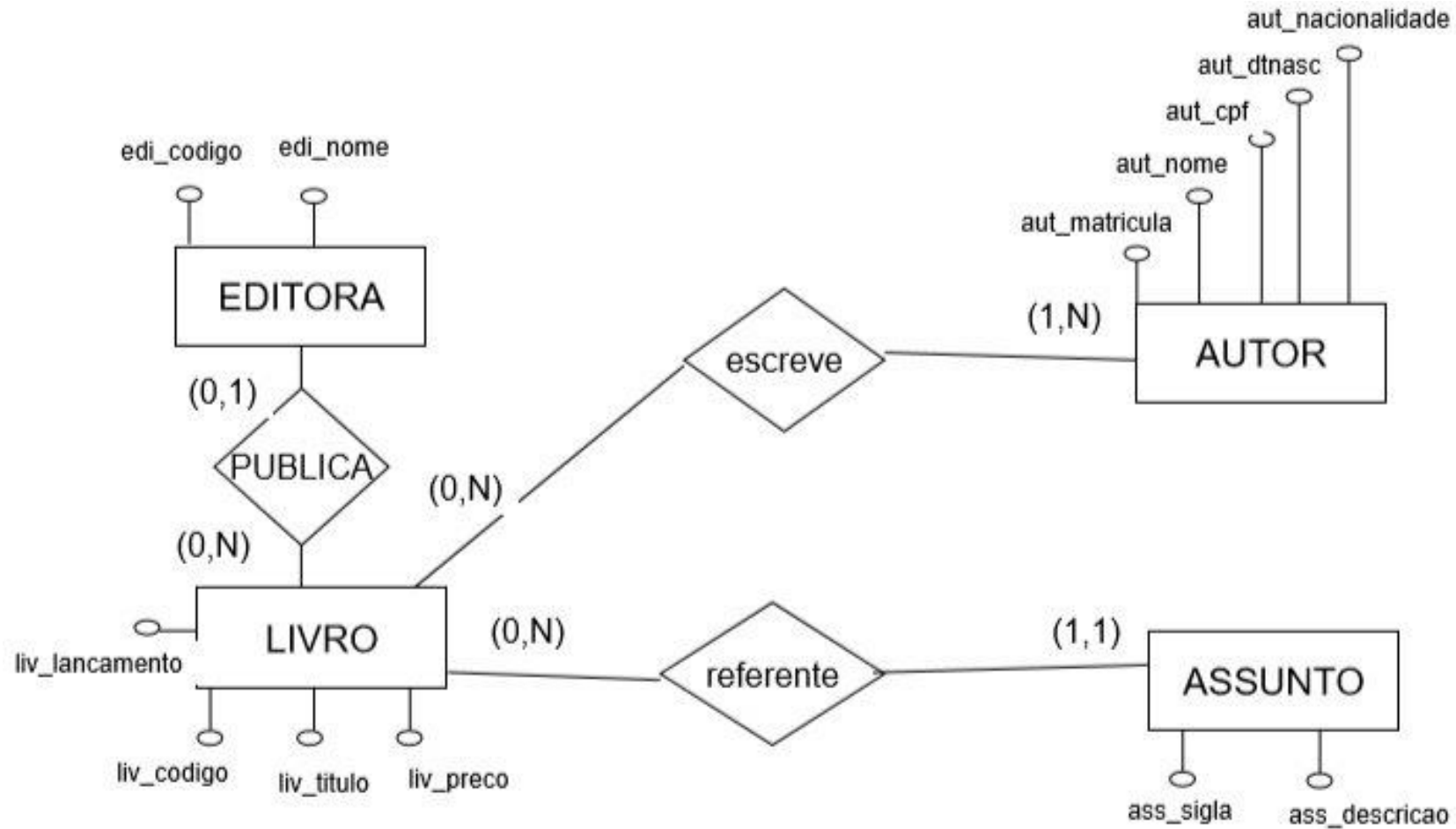
Cases de sucesso com uso do NoSQL



SGBD: Sistemas de processamento de ordem de venda.

NOSQL: Sistema de pesquisa, recomendações e adaptações de preços em tempo real.

Revisão - CREATE



Revisão - SELECT

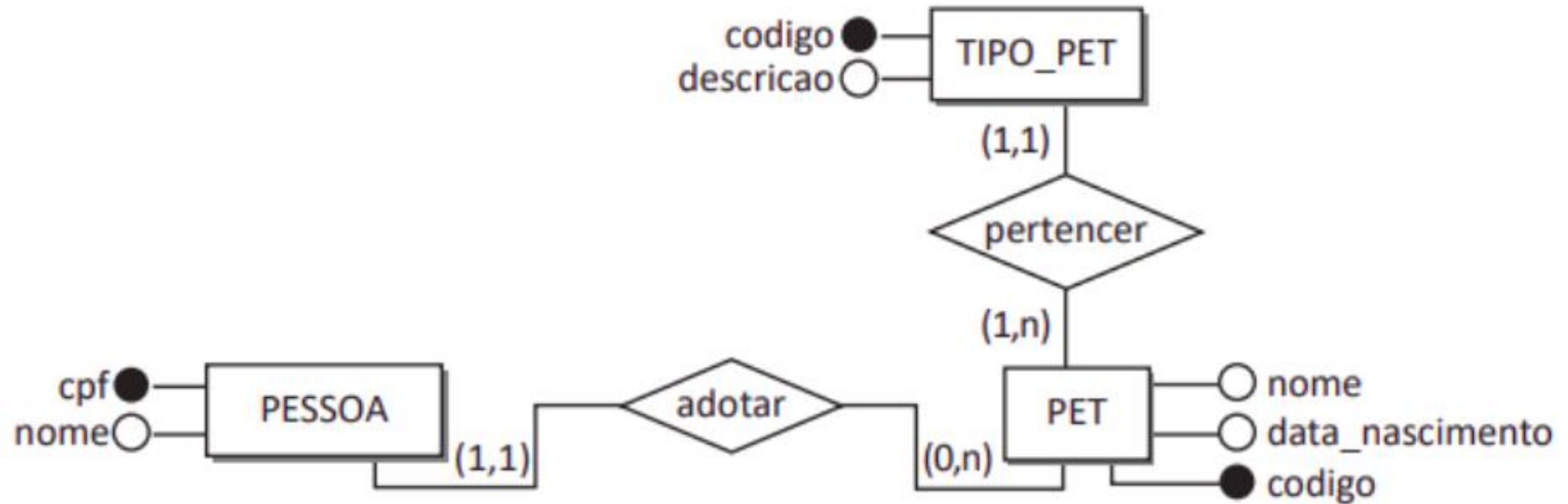
LIVRO						ESCREVE	
liv_codigo	liv_titulo	liv_preco	liv_lancamento	edi_codigo	ass_sigla	liv_codigo	aut_matricula
1	banco de dados para web	32,20	10/01/1999	1	BAN	1	1
2	programando em linguagem c	30,00	01/10/1997	1	PRO	2	1
3	progrmando em linguagem c++	115,50	01/11/1998	3	PRO	3	2
4	banco de dados na bioinformática	48,00		2	BAN	4	3
5	redes de computadores	42,00	01/09/1998	2	RED	5	4

ASSUNTO	
ass_sigla	ass_descricao
BAN	Banco de Dados
PRO	Programação
RED	Redes
SIS	Sistemas Operacionais

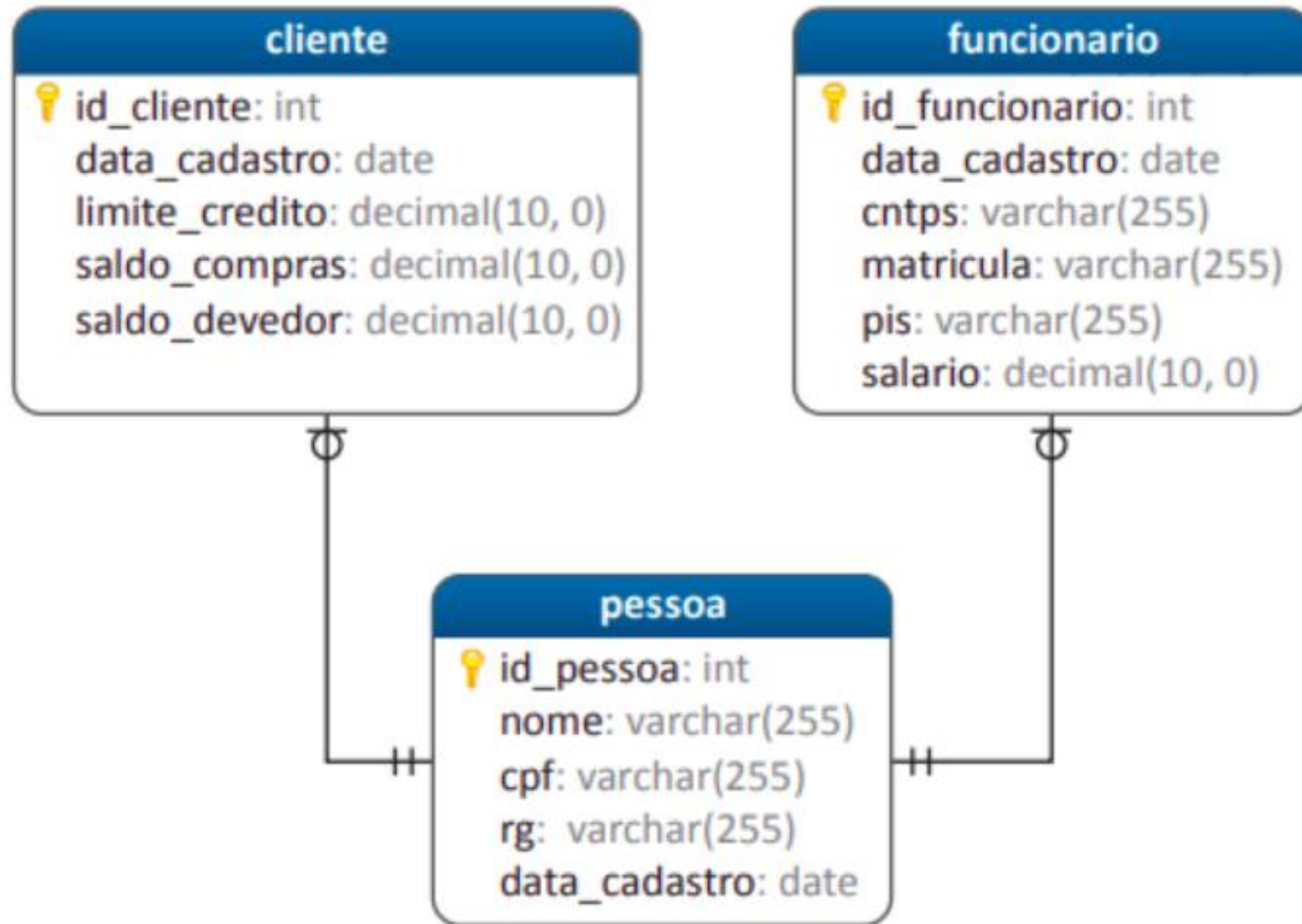
EDITORIA	
edi_codigo	edi_nome
1	Mirandela
2	Editora Via Norte
3	Editora Ilhas Tijucas
4	Maria José

AUTOR		
aut_matricula	aut_nome	...
1	Luiz	
2	Hugo	
3	Joaquim	
4	Regina	

Revisão - CREATE



Revisão - JOIN



Agradecimentos

**Slides baseados nas aulas dos professores Cristiane,
Claudio Benossi e Vanderson Bossi - Unicsul**